



Degree Project in Computer Science and Engineering

Second cycle, 30 credits

Multi-Node Attack and Defense Toolkit for Federated Learning

Exploration of Adaptive Aggregation Strategies

SIGVARD DACKEVALL

Multi-Node Attack and Defense Toolkit for Federated Learning

Exploration of Adaptive Aggregation Strategies

SIGVARD DACKEVALL

Degree Programme in Computer Science and Engineering

Date: July 22, 2025

Supervisors: Mauricio Byrd Victorica, , Salman Toor

Examiner: György Dán

School of Electrical Engineering and Computer Science

Host company: Scaleout Systems AB

Swedish title: Konstruktion av ett Multi-Nod attack och försvars-Toolkit för Federerad Inläring

Swedish subtitle: Utforskning av adaptiva aggregeringsmetoder

Abstract

Federated learning (FL) collaboratively trains models without centralizing raw data, however its openness for clients invites poisoning attacks that can degrade or hijack the global models. This thesis develops a Kubernetes-orchestrated, multi-node attack toolkit for FL that couples the production-grade FEDn framework with a library of pluggable adversaries and defenses. The platform scales from a handful to hundreds of containerized clients via a one-line change and supports realistic features such as Dirichlet non-IID data splits, late client joins, various adversary activation and finally mitigation strategies.

Using this test-bed, 180 controlled experiments were executed that crossed two attack families (static label-flipping and Little-Is-Enough), four data-partition regimes, two participation schedules, and three attacker densities (10%, 20%, 30%). In addition to a FedAvg baseline, this thesis benchmarks three state-of-the-art robust aggregation rules: Multi-KRUM, Trimmed-Mean, and Divide-and-Conquer. Furthermore, introduces EE-Trimmed-Mean, an adaptive rule that marries coordinate trimming with an ε -greedy trust score. The results confirm that no single static rule suffices across all conditions: Multi-KRUM can at times struggle to learn, whereas Trimmed-Mean does never fully exclude a malicious client update. EE-Trimmed-Mean achieved relatively good performance, while it stroke a balance of both including late benign client updates as well as filtering out malicious client updates.

Adaptive, aggregation is a promising path towards resilient FL, it preserves honest diversity while achieving high malicious exclusion, whilst not relying on hard static thresholds. The open-source toolkit released with this thesis lowers the barrier for rigorous, reproducible security research in federated learning and provides a launch-pad for future work in the field of improving robustness and privacy in FL.

Keywords

Docker Container, Kubernetes, Aggregation Rule (AGR), Federated Learning (FL), Combiner, Federated Average (FedAvg), Trimmed Mean (TrMean), KRUM, Multi-KRUM, Divide-and-Conquer (DnC)

Abstract

Federated learning (FL) möjliggör kollaborativ modellträning utan att centralisera rådata, men öppenheten för klienter inbjuder till förgiftningsattacker som kan försämra den globala modellen. Denna avhandling utvecklar ett Kubernetes-orkestrerat, flernodigt attackverktyg för FL som sammankopplar ett industriredo FEDn med ett bibliotek av pluggbara angripare och försvar. Plattformen kan skalas från ett fåtal till hundratals kontaineriserade klienter via en enda rad kod och stödjer realistiska funktioner såsom Dirichlet-baserade icke-IID-datasplitar, sena klientanslutningar, flera sorters attacker och en 4 robusta aggregeringsmetoder.

Med hjälp av denna testbädd genomfördes 180 kontrollerade experiment som använde två sorters attacker, från två olika attackfamiljer (label-flipping och Little-Is-Enough), fyra datapartitionsregimer, två deltagandescheman och tre angripartätheter (10%, 20%, 30%). Utöver FedAvg, som används som en baslinje under experimenten, jämför uppsatsen tre moderna robusta aggregeringsregler: Multi-KRUM, Trimmed-Mean och Divide-and-Conquer, samt introducerar EE-Trimmed-Mean, en adaptiv regel som förenar koordinattrimning med ε -greedy förtroendepoäng. Resultaten visar att ingen enskild statisk regel täcker alla förhållanden i studien. Multi-KRUM har svårigheter att generalisera, medan Trimmed-Mean aldrig helt utesluter en elakartad klient. EE-Trimmed-Mean uppnår relativt hög träffsäkerhet och balanserar dessutom förmågan att inkludera sena, godartade uppdateringar med att filtrera bort skadliga.

Adaptiv aggregering visar sig, som en lovande väg mot robust FL, både kan den bevara ärlig diversitet, utan de hårda och statiska trösklarna som är vanligt bland robusta aggregeringsmetoder. Vidare är simulatorn tillgänglig som ett öppen-källkodsverktyg som offentliggörs med denna avhandling. Vilket förhoppningsvis sänker tröskeln för säkerhetsforskning inom federerad inlärning och kan aggera som en språngbräda för framtida arbeten inom federerad inlärning.

Nyckelord

Dockerbehållare, Kubernetes, Aggregation regel (AGR), Federerad inlärning (FL), Combiner, Federated Average (FedAvg), Trimmed Mean (TrMean), KRUM, Multi-KRUM, Divide-and-Conquer (DnC)

Acknowledgments

I would like to express gratitude to my industry supervisor, Salman Toor at Scaleout Systems AB, throughout this project. Your support has enabled me to venture into new and unknown areas of federated learning research– something I am truly grateful for.

I also wish to thank my academic supervisor for helping me bring this thesis report to a respectable standard. Last, not least, to my mother, who has helped me with my applications for grants. At a time when my tuition had run out and I had no other source of income. On that note, finally my two grant providers: Svenska kyrkan in Burträsk and AAA for their grants, which made it possible for me to complete both this thesis and my Engineering degree.

Stockholm, July 2025

Sigvard Dackevall

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem and Research Questions	2
1.3	Purpose	3
1.4	Goals	3
1.5	Research Methodology	4
1.6	Sustainability and Ethics	6
1.7	Delimitations	7
1.7.1	Scope of Attacks and Experimental Variations.	7
1.8	Contribution	8
2	Background	9
2.1	Federated Learning	9
2.1.1	Vertical and Horizontal Federated Learning	10
2.1.2	Cross-Silo vs Cross-Device FL	10
2.1.3	Common Aggregation Rules	11
2.1.3.1	FedAvg	11
2.1.3.2	FedAdam	11
2.1.4	FEDn	13
2.2	Attacks against FL	14
2.2.1	Model Poisoning Attacks	15
2.2.1.1	Gradient Boosting Attacks	15
2.2.1.2	Little Is Enough Attack	15
2.2.2	Data Poisoning Attacks	16
2.2.2.1	Label Flipping Attacks	16
2.2.2.2	Backdoor Attacks	17
2.3	Datasets	18
2.3.1	IRIS	18
2.3.2	MNIST	18

2.4	Single Node Simulator	18
2.5	Mitigation Strategies	20
2.5.1	Trimmed Mean	20
2.5.2	KRUM and Multi-KRUM	21
2.5.3	Divide-and-Conquer	23
2.6	Exploration vs Exploitation	24
2.6.1	Epsilon-Greedy algorithm	25
2.6.2	Thompson Sampling	25
2.7	Related Work	25
2.7.1	Federated Learning Experimentation Frameworks and Simulators	26
2.7.2	Adversarial Attacks in Federated Learning	26
2.7.3	Exploration–Exploitation in FL Aggregation	27
3	Method	29
3.1	Research Process	29
3.2	Data Collection	32
3.2.1	Evaluation Scenarios	32
3.2.1.1	Scenario Dimensions	32
3.2.1.2	Public Dataset	33
3.2.1.3	Experiment Matrix	33
3.2.1.4	Practical Justification	33
3.3	Multi-Node Simulator Architecture	34
3.3.1	Multi-Node Simulator Overview	34
3.3.2	Containerization and Helm Chart Design	35
3.3.3	Data Partition and Distribution	36
3.3.3.1	IID label and volume data partitions	36
3.3.3.2	Non-IID client partitions	36
3.3.4	Client Lifecycle and Attack Execution	37
3.3.4.1	Label Flipping	38
3.3.4.2	Gradient Boosting	39
3.3.4.3	Little-Is-Enough	39
3.3.5	Mitigation Strategies	40
3.3.5.1	Trimmed Mean (TrMean)	40
3.3.5.2	KRUM	41
3.3.5.3	Multi-KRUM	42
3.3.5.4	Divide-and-Conquer (DnC)	42
3.4	Adaptive Mitigation Strategy	43
3.4.1	Exploration–Exploitation TrMean (EE-TrMean)	44

3.5	Experimental Setup	46
3.6	Evaluation framework	46
4	Results	48
4.1	Multi-Node FL Simulator	48
4.2	Experiments	51
4.2.1	Label flipping - IID Balanced	53
4.2.2	Label flipping - IID Imbalanced	56
4.2.3	Label flipping - non-IID Partially Imbalanced	58
4.2.4	Label flipping - non-IID Imbalanced	60
4.2.5	Little-is-Enough - IID Imbalanced	62
4.2.6	Little-is-Enough - non-IID Partially Imbalanced	64
4.2.7	Experiments and Research Questions	66
5	Discussion	67
5.1	Research Question 1: Realism and Scalability of the Simulator	67
5.2	Research Question 2: Behaviour of Existing Robust Aggregators	68
5.3	Research Question 3: Effectiveness of EE-Trimmed-Mean	69
6	Conclusions and Future Work	70
6.1	Limitations	70
6.2	Unaddressed Questions	71
6.3	Next Steps and Future Work	71
6.4	Reflections	72
	References	72

List of Figures

2.1	An overview of the FEDn architecture.	13
3.1	An overview of the simulation architecture.	34
1	Epsilon Decay and Alpha ramp-up	77
2	Multi-KRUM - non-IID - Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping Test Accuracy	77
3	Multi-KRUM - non-IID - Imbalanced - 3 Malicious Clients - 3 Malicious Clients 5 rounds late - Label Flipping - Timeline .	77
4	EETrMean - non-IID - Partially Imbalanced - 2 Malicious clients - 2 Benign clients 5 rounds late - Label Flipping - Test Accuracy	78
5	EETrMean - non-IID - Partially Imbalanced - 2 Malicious clients - 2 Benign Clients 5 rounds late - Label Flipping - Timeline	78
6	TrMean - non-IID - Partially Imbalanced - 2 Malicious clients - 2 Benign clients 5 rounds late - Label Flipping - Test Accuracy	78
7	TrMean - non-IID - Partially Imbalanced - 2 Malicious clients - 2 Benign Clients 5 rounds late - Label Flipping - Timeline . .	79
8	Fedavg - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds	79
9	Fedavg - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds	79
10	TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds	80

11	TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds	80
12	Multi-KRUM - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds	80
13	Multi-KRUM - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds	81
14	DNC - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds	81
15	DNC - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds	81
16	EE-TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds	82
17	EE-TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds	82
18	Fedavg - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds	82
19	Fedavg - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds	83
20	TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds	83
21	TrMean - non-IID - Partially Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds	83
22	Multi-KRUM - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds	84
23	Multi-KRUM - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds	84

24	DNC - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds	84
25	DNC - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds	85
26	EE-TrMean - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds	85
27	EE-TrMean - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds	85
28	Peak CPU consumption as the client count increases.	87
29	Peak memory consumption as the client count increases.	87

List of acronyms and abbreviations

AGR	Aggregation Rule
DLF	Dynamic Label Flipping (Attack)
DnC	Divide-and-Conquer
FedAdam	Federated Adam
FedAvg	Federated Averaging
FEDn	FL framework from Scaleout Systems
FEMNIST	Federated Extended MNIST
FL	Federated Learning
Git	Version control software
GitHub	Online repository
gRPC	Google Remote Procedure Call
Helm	A package manager for Kubernetes
IID	Independent and Identically Distributed
IoT	Internet of Things
IRIS	A classic small dataset
KRUM	A robust aggregator
KTH	KTH Royal Institute of Technology
Kubernetes (K8s)	A container orchestration platform
LIE	A Little Is Enough (Attack)
MNIST	Modified National Institute of Standards and Technology (Dataset)
Multi- Krum	A robust aggregator
non-IID	Non-Independent and Identically Distributed

OS	Operating System
PyTorch	A popular deep learning framework
S3	A cloud storage mechanism
SLF	Static Label Flipping (Attack)
SVD	Singular Value Decomposition
TensorFlow	A popular deep learning framework
TrMean	Trimmed Mean (A robust aggregator)
VM	Virtual Machine

List of Symbols Used

The following symbols will be later used within the body of the thesis.

- β The trimming fraction in Trimmed Mean, controlling how many largest/smallest values to remove.
- Δ_t The aggregated model update at round t , e.g., in FedAdam.
- $\Delta_{t,i}$ In FedAdam-like methods, the difference (update) between the local model on client i and the global model at round t .
- $\ell(w, b)$ A generic loss function on mini-batch b and model parameters w . Used during local training.
- η The (local) learning rate for gradient descent.
- $\hat{m}_t, \hat{v}_t$ Bias-corrected momentum and variance terms in the FedAdam optimizer.
- $\text{TrMean}(j)$ The trimmed mean for the j -th coordinate after discarding outliers.
- τ A small constant (epsilon) in FedAdam for numerical stability.
- B The mini-batch size used by each client for local training.
- C The set of selected clients in a given FL round, out of K total clients.
- $d(x_i, x_j)$ The Euclidean distance between update vectors x_i and x_j .
- E The number of local epochs each client performs before returning updates.
- f The maximum number (or fraction) of malicious clients that KRUM or Multi-KRUM can tolerate.

$L = \lfloor \beta N \rfloor$ Number of lowest-value coordinates discarded in Trimmed Mean.

m_t The total number of samples among selected clients at round t .

n_k The number of data samples possessed by client k . Used for weighted averaging (e.g., FedAvg).

S_i The Krum “score” of update x_i , computed by summing distances from x_i to its closest neighbors.

$U = N - \lfloor \beta N \rfloor$ Upper boundary for the highest-value coordinates in Trimmed Mean.

w_t The global model parameters at communication round t in a federated learning setup.

w_{t+1}^k The local model parameters for client k after it finishes local training at round $t + 1$.

x_{agg} or $\hat{x}_{\text{agg}}$ A final aggregated update in robust aggregators.

x_i A generic model-update vector from client i , used in robust aggregations.

Chapter 1

Introduction

1.1 Background

To our knowledge, one of the earliest mentions of Federated Learning (FL) can be found in the work of [1]. FL is a decentralized machine learning approach that enables model training across a distributed network of devices, called clients, without requiring direct access to their raw data. This approach allows data to remain on client devices, such as smartphones or IoT devices, while only client model updates are sent to a central server, where in turn the local client updates are aggregated using an Aggregation Rule (AGR), such as Federated Averaging (FedAvg), more mentioned subsequently on this below. A key advantage of FL is the decoupling of model training from data access, reducing privacy risks by limiting the attack surface to the device itself rather than to both the device and the cloud. In centralized settings, data breaches can expose large volumes of sensitive information, while in FL, raw data never leave the local device, minimizing the impact of server compromises [1].

However, FL introduces specific challenges compared to centralized methods. Data across different client devices are typically non-Independent and Identically Distributed (non-IID), meaning each client generates data reflecting unique user behaviors and patterns. This non-IID characteristic may lead to inconsistencies and biases in model updates. Another inherent challenge is data imbalance, where certain users contribute considerably more data than others, potentially skewing the aggregated global model [1].

Moreover, the distributed and decentralized nature of FL makes it vulnerable to adversarial scenarios, including malicious attacks designed to degrade the global model or compromise its integrity. Consequently, various defense techniques have been developed, aiming to identify and mitigate the

impact of adversarial activities, ensuring robust and reliable model training.

1.2 Problem and Research Questions

Although a wide range of FL attack and defense techniques has been proposed, evaluating these methods within realistic distributed environments remains challenging due to complexity, scalability, and reproducibility concerns. There is thus a clear need for a robust, scalable, and flexible simulation platform capable of emulating multi-node FL scenarios involving both honest and malicious participants under controlled, repeatable conditions.

This thesis addresses three complementary research questions:

Starting with the construction of a multi-node FL simulator supporting diverse attack scenarios and defense mechanisms, which facilitates systematic assessment of the attack and defense impact on FL training processes. Critically the multi-node simulator must be able to capture the essential distributed features of FL (a central server coordinating multiple clients performing local training), permit the injection of adversarial behaviors, integrate state-of-the-art mitigation strategies, and enable straightforward implementation and testing of new mitigation techniques. From this, the first research question is formulated:

- **Research Question 1:** How can a scalable, modular, and pluggable multi-node FL simulator be designed and implemented so that it supports realistic emulation of distributed training, enables controlled injection of adversarial behaviour and facilitates seamless integration of existing and novel defense mechanisms?

Current defense mechanisms often rely on static assumptions regarding client reliability, data distribution, and participation dynamics, potentially resulting in overly conservative strategies. These strategies risk excluding updates from honest yet late-joining clients or those with pronounced non-IID data partitions, consequently reducing diversity in aggregated updates and impairing the convergence and performance of the global model. This gives rise to two additional research questions:

- **Research Question 2:** To what extent do contemporary robust aggregation rules, such as Multi-KRUM, Trimmed Mean and Divide-and-Conquer, successfully discriminate between honest and malicious clients when the federation experiences non-IID data distributions and dynamic client participation?

- **Research Question 3:** How can we develop and validate an adaptive robust aggregation strategy that dynamically adjusts to variations in client reliability, data heterogeneity, and participation timing, while preserving resilience against adversarial attacks and maintaining global-model performance?

1.3 Purpose

Federated learning aims to leverage updates from as many honest client participants as possible to enhance global model performance. However, real-world FL deployments can face scenarios in which clients join training sessions at varying times due to connectivity issues, latency variations or intermittent availability. Additionally, FL inherently involves non-IID data distributions across clients, necessitating aggregation strategies specifically tailored to handle data heterogeneity [2].

The purpose of this thesis is to address these critical aspects by developing a controlled simulation environment, allowing systematic exploration and evaluation of adversarial dynamics in FL. A dedicated multi-node FL simulator provides a sandbox to safely study how different attack tactics can degrade model performance or leak sensitive data, and evaluate how effectively proposed defenses mitigate such threats. Furthermore, the development of a novel adaptive robust aggregation strategy will specifically address the limitations of current approaches by dynamically accommodating late-joining clients and diverse data distributions, crucial for practical FL deployment.

Ultimately, the motivation behind this work is to strengthen trust in FL by providing robust tools and methods that systematically enhance model resilience, guiding researchers and practitioners toward more secure and practical FL implementations suitable for privacy-sensitive and critical real-world applications.

1.4 Goals

The overarching objective of this thesis is to deliver a practical, research-grade testbed and accompanying methods that collectively strengthen the security and reliability of federated learning in the presence of adversarial behavior, dynamic client availability, and non-IID data.

To realize this objective, the work is organized around three concrete, measurable sub-goals:

1. Develop a multi-node simulator using Kubernetes, enabling the execution of existing attack experiments in a distributed environment.
2. Implement mitigation strategies (Multi-KRUM, Trimmed Mean and Divide-and-Conquer) and systematically evaluate their effectiveness under adversarial conditions and varying participation scenarios. Key evaluation metrics include both the inclusion and exclusion rates of late honest and late malicious clients respectively during the aggregation phase and the accuracy of the global model to gauge overall resilience against attacks.
3. Develop and evaluate one novel robust aggregation strategy that dynamically adapts to changes in client participation timing and non-IID data distributions. This goal is measurable by the mitigation strategies ability to increase the inclusion rate of late honest clients and maintain resilience against adversarial attacks compared to existing strategies, measured in global model test accuracy and exclusion rate of malicious client updates.

1.5 Research Methodology

This thesis adopts an experimental simulation-based methodology to investigate adversarial attacks and mitigation strategies in FL systems. The primary objective is to evaluate how both model poisoning and data poisoning attacks affect FL training and to assess the resilience of existing and novel AGRs in realistic, distributed settings. To achieve this, a pluggable multi-node attack simulator is developed on top of FEDn, a production-grade FL framework, allowing for flexible experimentation with various attack patterns, data distributions and client participation scenarios.

The research methodology is structured around the following components:

1. Literature Review: A comprehensive review is conducted to identify relevant adversarial attacks, mitigation strategies and robust aggregation rules applicable to FL. This ensures that the chosen methodologies and implemented defense mechanisms are grounded in existing research while also identifying gaps to be explored.

2. **Simulator Design:** A simulator environment is built to closely emulate real-world FL deployments. It includes multiple participating clients, both benign and malicious, communicating asynchronously with a central server via FEDn's multi-tier architecture. By employing containerized clients and orchestrating them in a Kubernetes cluster, the simulator enables realistic distributed training scenarios.
3. **Attack and Mitigation Implementation:** Two predefined adversarial strategies (e.g., label flipping and Little-is-Enough) are applied from client containers. These attacks can be triggered at different stages of training, including scenarios in which malicious clients appear late in the training session. In parallel, server-side defenses, including existing AGRs such as Trimmed Mean (TrMean), Multi-KRUM and Divide-and-Conquer (DnC), are deployed to test their ability to filter or mitigate these attacks.
4. **Dynamic Client Participation & Non-IID Data:** Experiments are configured to mirror realistic participation patterns and data distributions:
 - **Late-Joins:** Two late policies are enacted with either benign or malicious clients participating from the fifth round, reflecting network latencies or connectivity issues common in real FL systems.
 - **IID vs. Non-IID label distributions:** Both balanced and imbalanced data partitions are tested, capturing heterogeneity in client data. This setup emphasizes the aggregator's capacity to adapt under varying degrees of distribution skew.

By systematically manipulating these factors, the experiments reveal how existing AGRs and the newly proposed aggregation method handle dynamic participation and non-IID conditions without overly excluding honest clients.

5. **Evaluation Metrics & Comparative Analysis:** Performance is measured across a range of metrics:
 - **Exclusion and Inclusion Rates:** The proportion of late honest vs. malicious client updates accepted or rejected by the aggregator, indicating whether each defense strategy inadvertently filters beneficial updates or incorrectly includes malicious ones.

- **Global Model Accuracy:** Evaluating overall convergence and robustness against adversarial influence on the final aggregated model.
- **Convergence Behavior:** Analyzing training stability (e.g., losses and accuracy over communication rounds) under dynamic client participation.

Existing strategies (Multi-KRUM, Trimmed Mean, Divide-and-Conquer) are compared to one another, as well as to the novel adaptive approach, to determine effectiveness in realistic dynamic settings.

6. **Development of a Novel Aggregation Strategy:** Lastly, a new aggregator is implemented that explicitly accounts for dynamic participation timing and non-IID distributions, inspired by balancing exploration (accepting updates from uncertain or late clients) and exploitation (maintaining reliance on established benign clients). Its performance is benchmarked against the other AGRs, with a focus on reducing honest client exclusion rates and maintaining model resilience under adversarial conditions.

An experimental approach, rather than a purely theoretical analysis, is chosen because of the dynamic, stochastic nature of FL under adversarial conditions. Simulation-based experimentation enables controlled, yet flexible manipulation of attack and participation parameters, network configurations and mitigation policies. Although real-world deployment might introduce additional complexities, this simulation-driven approach closely approximates real conditions in a reproducible way. Ultimately, by applying this methodology, the study bridges the gap between academic literature on adversarial FL and practical system design, contributing insights for improving federated models resilience and security.

1.6 Sustainability and Ethics

Since all data comes from publicly available benchmark datasets (IRIS and MNIST) that contain no personally identifiable information, there are no direct privacy risks.

Hardening federated learning against poisoning attacks directly supports safer deployment of collaborative models in healthcare, finance and edge-IoT scenarios, settings where centralizing data is impractical or legally restricted.

By lowering the barrier to reproducing robust-aggregation research, the toolkit encourages inclusive participation from academics and small enterprises that lack large private datasets, fostering more equitable AI innovation.

1.7 Delimitations

This thesis focuses on the problem of adversarial data and model poisoning attacks within FL. Other threats, such as membership inference, fall outside the scope of this work. We concentrate on robust aggregation strategies (Multi-KRUM, Trimmed Mean and Divide-and-Conquer), along with our novel adaptive aggregator, rather than on security measures reliant on cryptographic protocols or differential privacy. Consequently, while these alternative defense paradigms are highly relevant to FL security, they demand fundamentally different setups and are deliberately excluded here.

All experiments are carried out in the constructed multi-node simulator environment, using the FEDn framework orchestrated with Kubernetes. This controlled setup allows for reproducible testing under carefully designed adversarial attacks but does not fully replicate highly heterogeneous network conditions. Additionally, although we explore dynamic client availability (late-joining clients) and multiple data distribution scenarios, we do not attempt continuous or long-term drift modeling. Finally, although we draw on reinforcement learning principles to guide our aggregator’s design, we limit ourselves to a proof-of-concept approach, rather than an extensive investigation of deep RL algorithms or parameter tuning. These delimitations ensure a focused scope: investigating how robust aggregation can effectively handle poisoning attacks under varying participation and data conditions, in a realistic yet controlled and reproducible environment.

1.7.1 Scope of Attacks and Experimental Variations.

To keep the work tractable, some limitations to the planned experiments were done. We compare multiple robust aggregators and the novel adaptive aggregation method on the MNIST model only, restricted to two attacks (label flipping and LIE), four data-split variations and two late-join scenarios at a fixed 10%, 20% and 30% malicious fraction.

Although a complete factorial exploration (including multiple malicious fractions, additional attacks and hyperparameter tuning) might offer a broader picture, it would exceed the available time constraints. These delimitations

attempt to strike a balance between methodological rigor and practical feasibility.

1.8 Contribution

In a recent work [2] several pitfalls in research on poisoning defenses for FL are highlighted. Such as relying on overly simplistic datasets that mask real-world heterogeneity, focusing on IID data splits that fail to reflect practical variability and testing defenses primarily against naive attacks rather than advanced or adaptive adversaries. These limitations can give an inflated impression of robustness when defenses are evaluated under unrealistically ideal conditions.

To address these challenges, this thesis makes several key contributions. First, it develops a multi-node FL simulator on top of the FEDn framework orchestrated with Kubernetes, enabling flexible orchestration of numerous client containers, both benign and malicious, that can join at varying times. This setup moves beyond single-machine simulations, providing a richer and more realistic environment.

Within this environment, three robust aggregation strategies: Trimmed Mean, Multi-KRUM and Divide-and-Conquer were implemented and tested under more demanding conditions than typical IID MNIST scenarios. These experiments, incorporate volume imbalanced data partitions and non-IID class imbalanced data partitioning. Doing so systematically addresses the need for more heterogeneous and advanced testing which is recommended in [2], providing a more rigorous basis for measuring each aggregator’s resilience.

Furthermore, this thesis presents a novel adaptive aggregator designed to accommodate dynamic client participation and data heterogeneity by balancing exploration and exploitation. While existing methods often rely on static assumptions that inadvertently might discard honest but “unusual” updates, the proposed approach seeks to reduce honest-client exclusion rates without undermining defense against powerful adversaries. In line with this goal, an expanded set of metrics, including the proportion of honest vs. malicious updates included vs. excluded, which provides a finer-grained insight into the aggregator’s ability to maintain both security and inclusivity.

By combining a production-grade distributed simulator with a thorough evaluation of existing and novel defense methods under realistic data and adversarial conditions, this thesis responds directly to calls for more rigorous, heterogeneous and advanced testing practices in FL security research, contributing toward more robust and trustworthy FL deployments in practice.

Chapter 2

Background

This chapter establishes the technical foundations on which the rest of the thesis builds. It first distinguishes horizontal from vertical FL, then cross-device from cross-silo deployments. It then surveys common aggregation rules that make collaborative training possible, introduces the FEDn framework that underpins the implementation work and reviews the principal data and model poisoning attacks that motivate robust aggregation. This chapter finishes by describing the datasets and the single-node simulator that constitute the starting point for the multi-node experiments developed later in the thesis.

2.1 Federated Learning

FL is a decentralized machine learning paradigm designed to collaboratively train models across multiple devices or entities without centralizing raw data. Initially proposed by [1], FL emerged primarily as a response to increasing concerns over data privacy, regulatory constraints and bandwidth limitations associated with centralized data collection. Under this approach, numerous client devices, such as smartphones, IoT sensors or institutional servers, individually train a shared global model on their local datasets. After local training, each client transmits its updated model parameters, rather than raw data, to a coordinating central server. This central server aggregates these updates using specific AGRs, such as the widely-used Federated Averaging (FedAvg) algorithm, to produce an updated global model [1].

2.1.1 Vertical and Horizontal Federated Learning

Federated Learning can be categorized into different types based on how data is distributed across clients. The two primary categories are Horizontal FL and Vertical FL, with the emerging area of Personalized FL addressing the challenges of adapting global models to heterogeneous client data [3].

Horizontal Federated Learning (HFL) also referred to as sample-based FL, the data is partitioned by samples, meaning different clients have data with the same feature space but belonging to different data subjects. For example, multiple hospitals may collect similar medical records, such as patient age, diagnosis and treatment outcomes, but each hospital serves different patients. Horizontal FL is commonly used when organizations or devices operate in similar domains but with different data points [3].

Vertical Federated Learning (VFL) or feature-based FL, assumes that clients share data about the same individuals or entities, but their datasets contain different feature spaces. For instance, a bank and an insurance company might both have information on the same customers, but the bank holds financial data while the insurer possesses health records [3].

This thesis will focus on the horizontal case of FL.

2.1.2 Cross-Silo vs Cross-Device FL

Federated Learning can furthermore be classified based on the scale and nature of participating clients into Cross-Device FL and Cross-Silo FL, each having distinct challenges and use cases [4, 3].

Cross-Device FL: This setting involves a large number of devices such as smartphones, IoT sensors or wearable devices, which contribute small amounts of data for model training. Cross-device FL faces challenges related to intermittent connectivity, limited computational power and energy constraints, as devices are often battery-powered and may have unreliable network access. Communication efficiency and failure resilience are critical in this scenario to ensure successful training rounds despite the highly dynamic environment [4, 3].

Cross-Silo FL: In contrast, Cross-Silo FL involves a smaller number of well-resourced entities, such as hospitals, banks or government institutions, that contribute large volumes of data for training. These clients typically have more stable infrastructure and computational capabilities, allowing for more complex models and longer training cycles. However, cross-silo FL may face regulatory challenges related to data sharing across organizations and require advanced security measures to protect sensitive information [4, 3].

2.1.3 Common Aggregation Rules

2.1.3.1 FedAvg

FedAvg combines locally trained models from distributed clients by averaging their updates in a weighted manner. Formally, FedAvg operates as follows. The server selects C clients out of K total clients to which the server will send the current global model w_t . The C clients will then perform multiple local updates based on their local and private dataset D_k^i . The local model is updated by minimizing a given loss function $\ell(w, b)$ through stochastic gradient descent over mini-batches of size B for E local epochs [1]:

$$w_{t+1}^k = w_t - \eta \sum_{i=1}^E \nabla \ell(w; b_i).$$

Once the local training is completed, the clients will send their updated models w_{t+1}^k back to the central server, where all of the model updates from each of the k selected clients of C will be aggregated using a weighted averaging approach based on the local data samples, n_k , available on each client [1]:

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{m_t} w_{t+1}^k \quad \text{where} \quad m_t = \sum_{k \in C_t} n_k.$$

This process of local training and global aggregation is repeated iteratively across multiple communication rounds. The training continues until a predefined number of rounds T is reached or until a convergence criterion, such as a target model accuracy or a reduction in the loss function, is satisfied [1].

2.1.3.2 FedAdam

FedAdam is an adaptive optimization method introduced for FL to address challenges posed by client heterogeneity and slow convergence, particularly in non-IID data settings. As an extension of the widely used FedAvg algorithm, FedAdam incorporates server-side adaptivity inspired by the Adam optimizer, which dynamically adjusts the global model updates based on momentum and variance scaling [5].

In FedAdam, participating clients receive the current global model and perform local training using stochastic gradient descent (SGD) on their private datasets. After completing local updates, clients compute their model updates $\Delta_{t,i}$, which represent the difference between their locally optimized model and

the received global model. These updates are sent to the central server, where they are aggregated into a single update Δ_t by averaging the updates across the participating clients [5]:

$$\Delta_t = \frac{1}{|S_t|} \sum_{i \in S_t} \Delta_{t,i},$$

where S_t represents the set of selected clients in the current round.

The distinguishing feature of FedAdam lies in how the server processes these aggregated updates. Instead of applying the updates directly to the global model, as in FedAvg, FedAdam employs the Adam optimization rule. This begins with the computation of the first moment (momentum) and second moment (variance) of the updates. The first moment is calculated as [5]:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \Delta_t,$$

where β_1 is the decay rate for momentum and m_{t-1} is the momentum estimate from the previous round. The second moment is updated using [5]:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \Delta_t^2,$$

where β_2 is the decay rate for variance and Δ_t^2 is the element-wise square of Δ_t . To correct for initialization bias in the early rounds, bias-corrected versions of these moments are computed as [5]:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \text{and} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

The global model is then updated using the bias-corrected moments, following the Adam rule [5]:

$$x_{t+1} = x_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \tau},$$

where η is the server learning rate and τ is a small constant introduced for numerical stability [5].

2.1.4 FEDn

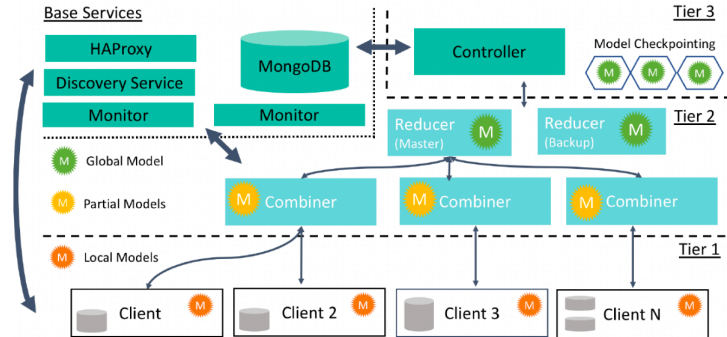


Figure 2.1: An overview of the FEDn architecture.

FEDn employs a tiered architecture designed to enable scalability, resilience and efficient model training across diverse FL use-cases. This architecture is divided into three tiers: clients (Tier 1), combiners and reducers (Tier 2) and the controller (Tier 3), each with distinct roles and responsibilities to support distributed learning effectively [3].

At Tier 1, clients are responsible for local model training on private data. These clients interact solely with assigned combiners, ensuring data privacy by restricting access to raw data. The black-box execution of the models of clients allows FEDn to remain agnostic to the underlying machine learning frameworks, with built-in support for PyTorch and TensorFlow [3].

Tier 2 comprises of combiners and a reducer, which form the backbone of FEDn’s scalability and robustness. Combiners are stateless gRPC servers tasked with aggregating model updates from a subset of clients into partial model updates. Their stateless nature allows for replacement in the event of failure, ensuring fault tolerance. The workload of each combiner scales linearly with the number of connected clients, enabling horizontal scalability by adding more combiners to the network [3].

The reducer is responsible for consolidating the partial model updates from all combiners into a unified global model. Operating in an active-passive configuration, the reducer setup reduces the risk of a single point of failure [3].

Tier 3, the controller, oversees the overall coordination of training rounds through a round participation policy. This policy includes steps such as checking combiner readiness, setting deadlines for partial updates, validating the global model and maintaining a trail of model checkpoints. The controller

manages consistency and adds robustness by handling failures, such as reassigning clients to new combiners or resuming training from the last completed round in case of disruptions [3].

2.2 Attacks against FL

Attacks against FL networks can be categorized primarily into two types: data poisoning and model poisoning attacks [4], [6]. Data poisoning attacks involve adversaries manipulating the local datasets of participating clients to indirectly degrade or bias the global model. Examples of such attacks include label flipping and backdoor attacks. In contrast, model poisoning attacks directly alter the model updates that clients send to the central aggregator, with the intent to degrade global model performance or inject biases directly into the model [4], [6].

These attacks can further be divided into targeted and untargeted categories. Targeted attacks, such as backdoor attacks, aim to compromise the global model's performance on specific inputs, causing intentional misclassification. Untargeted attacks, however, seek to reduce the overall accuracy of the model or introduce biases across all predictions indiscriminately [7].

Furthermore, attacks can also be classified based on the adversary's knowledge about the global model. In a white-box scenario, the adversary has complete access to global model parameters and outputs, enabling highly optimized attacks. Conversely, a no-box scenario assumes the adversary has no direct access to model parameters, instead relying on indirect methods such as poisoning local data. The no-box setting is particularly relevant in cross-device FL environments, where adversaries often have limited interaction with the global model [4].

Additionally, another important category of attacks are inference attacks, which targets privacy rather than model quality. Inference attacks, attempts to analyze the gradients or model updates passed to the server to extract information about the clients local data. For example in recent work [8], it is shown that an honest-but-curious server can analytically (or numerically) recover each client's label distribution and even exploit that knowledge to accelerate training [8]. Model-inversion attacks attempts to go further, by reconstruct the actual data points. In a recent work, it is shown that one can reconstruct high-fidelity ImageNet-size images from a single round of shared model updates [9]. However, these attacks represent a fundamentally different privacy threat and are beyond the scope of detailed discussion in this paper, which focuses on poisoning attacks and mitigation.

2.2.1 Model Poisoning Attacks

2.2.1.1 Gradient Boosting Attacks

The gradient boosting attack, as outlined in [7], is a targeted model poisoning technique that leverages the FL aggregation process to maximize an adversary's influence over the global model. Both FedAvg and FedAdam update the clients scaled on their local dataset size. The authors in [7] estimate the global model scaling factor and apply an explicit boosting factor, which is the inverse of the scaling factor used during aggregation. This adjustment effectively neutralizes the dilution effect caused by the aggregation step, allowing the attacker's update to fully dictate the model's learning trajectory [7].

Mathematical formulation: Let x_k be the update produced by malicious client k after local training and let $\alpha_k = \frac{n_k}{\sum_{j \in S_t} n_j}$ be the weight FedAvg (or FedAdam) would apply during aggregation. The attacker counters this dilution by transmitting a boosted update

$$\tilde{x}_k = \frac{1}{\alpha_k} x_k,$$

so that, after the server's weighted average, $\alpha_k \tilde{x}_k = x_k$ and the malicious contribution is restored to full strength. More aggressive variants replace $1/\alpha_k$ with a constant boost factor $B > 1$ (e.g. 10 or 100) to dominate honest updates outright.

2.2.1.2 Little Is Enough Attack

The A Little Is Enough (LIE) attack is a stealthy model poisoning attack in FL that prevents model convergence and attempts to bypass robust AGRs such as Trimmed Mean and KRUM. Unlike traditional attacks that introduce extreme outliers, LIE subtly perturbs model updates within an undetectable range, making it difficult for defenses to filter out malicious updates. The attack assumes knowledge of the AGR, model architecture and statistical properties of client updates, making it an AGR-specific white-box attack [10].

LIE assumes client model updates follow a normal distribution. By estimating the mean (μ) and standard deviation (σ) of updates, the attacker introduces small systematic deviations within an estimated accepted range based on the estimated μ and σ . The poisoned updates thus remain statistically indistinguishable from benign ones, allowing the attack to evade detection while gradually steering the model away from its optimal convergence

path [10].

Once an accepted range of perturbations is determined, the attacker adapts its strategy based on the specific AGR in use. For Trimmed Mean, the attack ensures updates remain within the trimmed range. Against KRUM, malicious updates are slightly modified to appear closest to benign updates, increasing the likelihood of selection [10].

The effect is particularly severe in non-IID settings, where variations in update distributions complicate distinguishing natural client heterogeneity from adversarial manipulation [10].

Mathematical formulation: Assume benign updates $x_1, \dots, x_H \in R^d$ are IID with coordinate-wise mean μ and standard deviation σ (both estimated by the adversary from past rounds). For each malicious client m the attacker crafts

$$\tilde{x}_m = \mu + \gamma \sigma \odot \mathbf{s},$$

where $\odot$ denotes the element-wise product, γ is a small constant (e.g. $\gamma = 0.1$), and $\mathbf{s} \in \{-1, +1\}^d$ chooses the sign that most increases loss on a target example or pushes the model away from convergence. Because each coordinate lies within $[\mu^{(j)} - \gamma\sigma^{(j)}, \mu^{(j)} + \gamma\sigma^{(j)}]$, $\tilde{x}_m$ masquerades as a statistically plausible benign update and slips past outlier-based defenses such as TrMean or KRUM [10].

2.2.2 Data Poisoning Attacks

2.2.2.1 Label Flipping Attacks

Label flipping attacks are data poisoning attacks in FL where adversaries systematically mislabel training samples to degrade the global model performance. Unlike model poisoning attacks, which directly alter model updates, label flipping attacks operate at the data level, modifying local datasets without requiring direct interference with training. This makes them particularly relevant in cross-device FL settings, where adversaries may have limited access to gradients or aggregation rules but can manipulate local data before training. The core objective of an label flipping attack is to induce systematic misclassification or increase divergence between benign and poisoned updates, causing targeted or untargeted model degradation [4].

There are two types of label flipping attacks [4], static label flipping (SLF) and dynamic label flipping (DLF), differing in the adversaries knowledge and attack methodology. SLF is a black-box attack; where the adversary flips labels according to a predefined scheme, such as swapping class y_1 with y_2 in a binary

classification task.

DLF is a white-box attack where the adversary uses a surrogate model to identify the least probable correct label for each data sample and mislabels it to maximize misclassification. Leveraging knowledge of global model decision boundaries, DLF crafts more targeted and effective poisoning strategies, resulting in a more damaging attack than SLF when model access is available [4].

Mathematical formulation: Let the true label space be $\mathcal{Y} = \{0, 1, \dots, L - 1\}$. The static rule $\phi : \mathcal{Y} \rightarrow \mathcal{Y}$ used in this thesis is

$$\phi(y) = (y - l - 1) \bmod L,$$

with shift parameter $l = 0$. During local training a malicious client replaces every labelled example (x, y) in its dataset by $(x, \phi(y))$. When a fraction p of clients apply ϕ , the global risk is minimised at a biased classifier that confuses the L -class decision boundary [4].

2.2.2.2 Backdoor Attacks

Robustness threats include targeted backdoor triggers, posing significant risks to global model integrity [6]. Recent research by [11] investigates backdoor attacks in FL, emphasizing factors influencing success, such as the fraction of compromised clients, complexity of backdoor tasks and model capacity. Their findings show that higher numbers of backdoor tasks increase the difficulty of maintaining performance simultaneously. They demonstrate norm-clipping and weak differential privacy significantly mitigate attack impacts without severely degrading model performance.

The study introduces two primary attack models: fixed-frequency attacks, where adversaries participate at predetermined intervals and random sampling attacks with stochastic adversary participation. Results indicate fixed-frequency attacks are generally more effective due to their predictability, whereas random sampling results in variable efficacy. Backdoor effectiveness diminishes when malicious client proportion falls below 1%, which highlights the difficulty in executing successful attacks in large-scale FL environments with random client selection [11].

These insights emphasize the importance of adaptive defense mechanisms, like robust aggregation combined with privacy-preserving techniques, to counter evolving backdoor threats in FL [11].

Mathematical formulation: Let $t \in R^{d_{\text{in}}}$ be a small, fixed trigger pattern and $1_{\mathcal{T}}$ the binary mask that selects trigger pixels. For any clean input x the

adversary constructs

$$x^{\text{bd}} = (1 - 1_{\mathcal{T}}) \odot x + 1_{\mathcal{T}} \odot t, \quad y^{\text{bd}} = y_{\star},$$

where $y_{\star}$ is the attacker-chosen target label. During local training each malicious client replaces a proportion ρ of its samples by $(x^{\text{bd}}, y_{\star})$. After enough poisoned rounds the global model learns a decision function $\hat{f}$ such that $\hat{f}(x^{\text{bd}}) = y_{\star}$ while preserving high accuracy on benign data x [11].

2.3 Datasets

2.3.1 IRIS

The IRIS dataset is a classic small dataset for pattern recognition, consisting of 150 samples of iris flowers with four features each, categorized into 3 species (classes) with 50 samples per class [12]. Due to its limited size, IRIS is not included in major FL benchmark suites, but it does appear in tutorial examples for FL libraries (e.g., the Flower framework) [13]. A typical federated setup simulates 3 clients (one per species) with 50 samples each, mirroring the dataset’s balanced class distribution.

2.3.2 MNIST

The MNIST dataset is a widely used benchmark of handwritten digit images, consisting of 70,000 grayscale 28×28 images of the digits 0 through 9 (10 classes in total) [14]. Although not originally created for FL, MNIST has become a standard baseline in FL research and is featured in many FL frameworks and studies; for example, the federated averaging (FedAvg) algorithm was first demonstrated on MNIST classification [13]. The full MNIST dataset contains 70,000 samples evenly spread across 10 classes (roughly 7,000 images per digit) [14]; in a common FL experiment with on the order of 100 clients, this translates to roughly 600–700 images per client on average, though the exact per-client sample counts and class breakdown depend on the chosen partitioning strategy.

2.4 Single Node Simulator

Since this work builds upon previous research by [15], this section details the methods it employed. The single-node simulator investigated the vulnerability

of FL models to adversarial behaviors by employing two core machines: one for hosting all server-side FEDn components and one for managing multiple simulated clients. On the server machine, FEDn's controller, reducer and combiner processes were located to orchestrate the overall federated workflow and aggregation. The controller acts as a coordinator for training rounds, distributing an initial seed model and checkpointing intermediate models as training progressed. Meanwhile, the reducer and combiner jointly handled the aggregation of client updates, with the combiner forming partial aggregates and the reducer merging these aggregates into a global model.

On the client machine, the user could create multiple FL clients, both benign and malicious. Although the clients were separate, all the clients shared both the operating system, networking interface and geographical position, thereby reducing the complexity of simulating network variability of a distributed network. Each client retrieved a partition of the dataset stored on this same machine, loaded a local version of the global model, performed training (whether benign or adversarial) and then sent parameter updates back to the combiner. This setup relied on a Python-based simulation script, launched through a virtual environment, which specified the number of clients, the fraction of malicious clients, relevant hyperparameters and the FEDn server's IP address. The script automated the client initialization and round-by-round interactions, making it straightforward to run a suite of experiments.

The training session begins when the server's controller broadcast a seed model to all connected clients. Once training is started, benign clients performed conventional local learning steps on their assigned data, while malicious clients introduced one of the attacks implemented, namely label-flipping, gradient boosting, backdoor or LIE. After the clients finish the local training for a specified number of epochs, each client submits its updated parameters back to their designated combiner, which aggregates them into a partial update. The reducer then aggregates every partial update sent to it into an updated global model and in return sends it back to the clients for the next round. Concurrently, clients gather metrics such as local accuracy or loss and in many experiments, these values are then sent back to the server for logging and downstream analysis. This iterative procedure continues for a fixed number of rounds.

2.5 Mitigation Strategies

FL requires aggregating locally trained model updates from numerous clients, some of which may behave maliciously or exhibit faulty behavior. A naive averaging scheme, such as FedAvg, proves vulnerable to outliers: a small subset of adversarial clients can submit corrupted gradients that disproportionately skew the global model. To address this threat, researchers have introduced AGRs, which non-linearly filter or reduce the impact of suspicious updates before combining them [16].

Early work drew on robust statistics, exemplified by the trimmed mean and coordinate-wise median, which mitigate extreme values on a per-dimension basis [16, 17]. Further developments emphasized the geometry of high-dimensional updates. For instance, KRUM identifies the most “central” gradient by comparing pairwise euclidean distances, thereby it discards outliers [17]. More advanced or hybrid methods, such as Bulyan [18] or Divide-and-Conquer [19], combine filtering strategies with robust averaging to further guard against sophisticated attacks.

Although these approaches differ in detail, ranging from coordinate trimming to spectral analysis, they share the core objective of preserving signals from the majority of honest clients while removing malicious or anomalous updates. In effect, robust AGRs act as a defensive layer, ensuring that even if a certain fraction of participants collude or fail arbitrarily, the global model remains accurate and stable.

The following subsections will highlight some of the common mitigation strategies.

2.5.1 Trimmed Mean

To improve robustness against Byzantine adversaries, Trimmed Mean (TrMean) was developed, a Byzantine-resilient AGR designed to mitigate the impact of malicious or extreme updates. In this approach, each client submits its model update to the server. In FEDn, these updates are first processed by combiners before being passed to the reducer. Instead of averaging all received updates directly (as in FedAvg), Trimmed Mean aggregates each coordinate of the parameters independently, applying a coordinate-wise sorting and filtering mechanism to remove outliers [16].

Consider a setting with N clients participating in a session round, each

producing an update vector:

$$x_1, x_2, \dots, x_N \quad \text{where} \quad x_k \in R^d.$$

The objective is to combine these N vectors into a single aggregated update,

$$x_{\text{agg}} \in R^d.$$

Trimmed Mean (TrMean) is implemented coordinate-wise. Specifically, for each dimension $j \in \{1, 2, \dots, d\}$, the process extracts the j -th coordinate from all update vectors,

$$x_1^{(j)}, x_2^{(j)}, \dots, x_N^{(j)},$$

then sorts these values in ascending order:

$$x_{(1)}^{(j)} \leq x_{(2)}^{(j)} \leq \dots \leq x_{(N)}^{(j)},$$

where $x_{(k)}^{(j)}$ denotes the k -th smallest value in the j -th dimension.

A trimming fraction β (with $0 < \beta < 0.5$) specifies how many of the lowest and highest values to remove. Let

$$L = \lfloor \beta N \rfloor \quad \text{and} \quad U = N - \lfloor \beta N \rfloor.$$

The lowest L and highest L values are discarded, retaining only the middle set

$$x_{(L+1)}^{(j)}, x_{(L+2)}^{(j)}, \dots, x_{(U)}^{(j)}.$$

The trimmed mean for each dimension is then calculated as:

$$\text{TrMean}^{(j)} = \frac{1}{U - L} \sum_{k=L+1}^U x_{(k)}^{(j)}.$$

Performing this procedure across all dimensions yields the aggregated update vector:

$$x_{\text{agg}} = (\text{TrMean}^{(1)}, \text{TrMean}^{(2)}, \dots, \text{TrMean}^{(d)}).$$

2.5.2 KRUM and Multi-KRUM

KRUM was developed to enhance robustness against Byzantine adversaries in distributed learning and FL. The core idea behind KRUM is to identify the most central client update, i.e., the one that deviates the least from its

closest neighbors. However, an important limitation is that KRUM was developed under the assumption of IID client data, which does not necessarily hold in practical FL scenarios where client datasets are often non-IID. This assumption can significantly impact KRUM's effectiveness in real-world settings [17].

The first step in KRUM is to compute all pairwise Euclidean distances between client updates. This allows the algorithm to assess the relative similarity between updates. Following this, KRUM excludes $f+2$ updates before computing scores, where f is a predefined threshold representing the maximum number of Byzantine (malicious) clients the system assumes it can tolerate. This filtering step ensures that extreme outliers, which could be adversarial updates, do not influence the final selection [17].

Once outliers are removed, KRUM computes a KRUM score for each remaining update. The score is calculated as the sum of the squared Euclidean distances to its $n-f-2$ closest neighbors. Finally, the update with the lowest score is selected as the global model update, as it is considered the most representative of the majority of honest clients [17].

Multi-KRUM which is essentially the same AGR method, but once the outliers have been disregarded. The algorithm chooses the lowest k scores of the updates remaining, where they state that m can be chosen according to robustness vs convergence. Following this they apply the standard FedAvg to compute the average of the updates with best KRUM scores [17].

Consider a scenario with N clients each producing a model update

$$x_i \in R^d$$

during a session round.

When using KRUM, each aggregation step proceeds as follows. Let x_i be a model update under consideration at a given tier. For each x_i , compute its Euclidean distance to every other update x_j :

$$d(x_i, x_j) = \|x_i - x_j\|_2.$$

f specifies the maximum number of tolerated malicious (or adversarial) clients. For each update x_i , sort the distances $\{d(x_i, x_j)\}$ in ascending order and sum the closest $N - f - 2$ distances:

$$S_i = \sum_{j \in \text{the } (N-f-1) \text{ closest to } x_i} d(x_i, x_j).$$

A smaller S_i indicates that x_i is more consistent with the majority of the other updates (i.e., it is closer to them in the parameter space).

Finally KRUM selects the update x_k with the minimum score S_k among all $\{x_i\}$. This selected update x_k is then used as the aggregated result:

$$x_{\text{agg}} = x_k.$$

Multi-KRUM selects multiple updates with lowest KRUM scores and aggregates them using FedAvg to balance robustness and convergence [17].

2.5.3 Divide-and-Conquer

Divide-and-Conquer (DnC) is designed to mitigate adversarial attacks in FL by filtering out malicious model updates before aggregation. In each communication round, participating clients submit model updates, each consisting of a high-dimensional parameter vector. Due to the computational complexity of applying spectral analysis directly to the full parameter space, DnC first performs random subsampling, selecting a subset of dimensions to reduce the effective dimensionality. This ensures that the method remains computationally feasible while still capturing meaningful variations across clients [19].

To isolate deviations, the algorithm computes the mean across clients for each selected dimension and centers the data by subtracting this mean, forming a matrix of centered updates. Singular Value Decomposition (SVD) is then applied to this matrix to identify the principal directions of variance among client updates. The top right singular vector, which corresponds to the largest singular value, reveals the direction in which updates deviate the most. Each client's projection onto this singular vector is then used to compute an outlier score, quantifying how far the client's update lies from the majority distribution. Clients with the highest outlier scores are considered potential adversaries and their updates are removed from aggregation [19]. To enhance robustness, the process is repeated multiple times with different randomly selected subsets of dimensions. In each iteration, the clients whose updates remain after filtering are stored in a set of "good" updates. Only these clients' updates are then aggregated using FedAvg [19].

In round t the server (or a combiner) receives N client updates $x_1, \dots, x_N \in R^d$. The server chooses a sub-dimension $d' \ll d$ and draws a random index set $\mathcal{I} \subset \{1, \dots, d\}$ with $|\mathcal{I}| = d'$. Next, the server forms the

centred matrix

$$\mathbf{Z} = [x_1^{(\mathcal{I})} - \bar{x}, \dots, x_N^{(\mathcal{I})} - \bar{x}]^\top \in R^{N \times d'}, \quad \bar{x} = \frac{1}{N} \sum_{i=1}^N x_i^{(\mathcal{I})}.$$

Let $u \in R^{d'}$ be the (unit-norm) first left-singular vector of $\mathbf{Z}$ (computed via any black-box SVD/PCA routine). The server projects each update and obtains the scores:

$$s_i = |u^\top (x_i^{(\mathcal{I})} - \bar{x})|, \quad i = 1, \dots, N.$$

Given a tolerance f (maximum Byzantine count), the server keeps the $N - f$ clients with the smallest scores s_i and disregard the rest. The server repeats the above sub-sampling–projection step R times with independent $\mathcal{I}_r$. A client survives DnC only if it is never filtered out across all R runs; denote the survivor set by $\mathcal{G}$.

Finally, the server applies FedAvg over the good updates:

$$x_{\text{agg}} = \frac{1}{\sum_{i \in \mathcal{G}} n_i} \sum_{i \in \mathcal{G}} n_i x_i.$$

Intuitively, the leading-singular direction u reveals the axis of largest variation; malicious updates that deviate the most from the majority score high on $|u^\top(\cdot)|$ and are pruned, while honest but merely noisy updates tend to cancel out across multiple random subspaces [19].

2.6 Exploration vs Exploitation

One of the core ideas within reinforcement learning is the concept of Exploration vs. Exploitation. The idea is founded on balancing exploration of potentially rewarding but untested actions and exploitation of actions already known to yield high returns [20, 21]. Exploration essentially means that the agent wants to gather information about the environment to ensure it is leveraging a good action at a given situation [22]. Exploitation, on the other hand, is where the agent leverages it's gathered knowledge about the environment to maximize rewards, by taking the most promising actions. The multi-armed bandit framework visualizes this idea: each “arm” (imagine a slot machine arm) has an unknown reward distribution and the agent must sequentially decide which arm to pull to maximize cumulative reward.

2.6.1 Epsilon-Greedy algorithm

Formally, let K denote the number of arms (actions) and let $r_t \in R$ be the reward obtained on the t -th play. The ϵ -greedy policy maintains an empirical mean estimate $\hat{\mu}_a(t)$ for each arm $a \in \{1, \dots, K\}$, computed as [21]:

$$\hat{\mu}_a(t) = \frac{1}{N_a(t)} \sum_{\tau \in I_a(t)} r_\tau,$$

where $I_a(t) \subseteq \{1, \dots, t-1\}$ are the time steps when arm a was pulled and $N_a(t) = |I_a(t)|$. At decision step t , the policy chooses

$$a_t = \begin{cases} \arg \max_{a'} \hat{\mu}_{a'}(t), & \text{with probability } 1 - \epsilon, \\ \text{an arm uniformly at random,} & \text{with probability } \epsilon. \end{cases}$$

The idea is that this balances the exploitation of the best estimate $\arg \max_{a'} \hat{\mu}_{a'}(t)$ against the exploration of other arms with probability ϵ .

2.6.2 Thompson Sampling

Thompson Sampling takes a Bayesian perspective by maintaining a posterior distribution over each arm's reward parameter. For example, consider a Bernoulli bandit with unknown probability of success $\theta_a \in [0, 1]$ for each arm a . We place an independent Beta prior $\text{Beta}(\alpha_a, \beta_a)$ on each θ_a . After observing a reward $r_t \in \{0, 1\}$ for arm a_t , the posterior for θ_{a_t} is updated to $\text{Beta}(\alpha_{a_t} + r_t, \beta_{a_t} + 1 - r_t)$. The action selection at step t is then [23]:

1. Sample $\tilde{\theta}_a(t)$ from the current posterior of each arm a .
2. Choose $a_t = \arg \max_a \tilde{\theta}_a(t)$.

This procedure, also called posterior sampling or probability matching, automatically balances exploration and exploitation: arms with higher (posterior) expected reward tend to be sampled more often, yet arms with high uncertainty still have a chance to appear optimal when sampled [24, 25].

2.7 Related Work

This chapter reviews prior work along three strands that directly motivate the thesis. First, we examine federated-learning toolkits and simulators,

contrasting early benchmark suites with newer multi-node frameworks. Second, we summarise the current landscape of adversarial attack studies—emphasising how evaluation setups such as [15] fall short of real-world conditions and how our simulator closes that gap. Third, we survey exploration–exploitation methods that adaptively choose or weight clients, laying the conceptual groundwork for the adaptive aggregator proposed later.

2.7.1 Federated Learning Experimentation Frameworks and Simulators

Several frameworks have been developed to support experimentation and deployment of FL. LEAF [26] is an early benchmark suite that provides standardized datasets and evaluation tools tailored to FL’s unique challenges, such as statistical heterogeneity and varying client participation. While valuable for benchmarking, LEAF does not natively support adversarial scenarios or robust aggregation evaluations.

More recently, FEDn [3] has emerged as a production-grade framework designed for scalability and modularity. Its tiered architecture supports realistic multi-node deployments and allows for flexible integration of custom aggregation logic and adversarial strategies, making it a suitable foundation for the simulator developed in this thesis. In parallel, frameworks like FedML and its extension FedMLSecurity have introduced built-in support for simulating a wide array of attacks and defenses, further highlighting the growing emphasis on adversarial robustness in federated environments.

In contrast to LEAF and general-purpose FL libraries, this thesis presents a scalable, Kubernetes-based multi-node simulator built on top of FEDn, explicitly targeting adversarial experimentation and robust aggregation evaluation.

2.7.2 Adversarial Attacks in Federated Learning

A growing body of literature investigates adversarial vulnerabilities in FL. [7] demonstrate that carefully crafted model poisoning updates can lead to targeted misclassifications even under robust aggregation, challenging the assumption that such defenses are sufficient. Similarly, [10] proposes Little Is Enough (LIE) attack, which introduces small, statistically plausible perturbations to circumvent outlier-based defenses such as Trimmed Mean and KRUM.

In recent work [19], a general optimization framework is presented that

tunes poisoning strategies for maximal damage across various aggregation rules. Their results show that popular robust AGRs may be significantly more susceptible to adaptive model poisoning than previously thought, particularly in non-IID settings. These findings underscore the need for evaluating FL systems against stronger, more sophisticated adversaries, especially in realistic heterogeneous conditions, the objective directly aligns with the simulation capabilities built in this thesis.

Earlier work by [15] introduced a single-node FEDn-based test-bed that could enabled four common poisoning attacks. While valuable for proof-of-concept studies, its co-location of all clients on one physical host imposed three key constraints: (i) scalability, client count was limited by a single machine’s resources; (ii) network realism, all updates travelled over the loopback interface, eliminating latency variance that often influences convergence and defence behaviour; and (iii) reproducibility, the Python-virtual-env setup was sensitive to library versions and diverged from the latest FEDn release. The multi-node Kubernetes simulator proposed in this thesis addresses these shortcomings by containerising each client, orchestrating them across a cluster, and exposing them to genuine inter-node network conditions. This design choice enables evaluation of robust aggregation rules under realistic communication delays, dynamic client (dis)connections, and larger-scale adversarial participation scenarios that could not be explored in [15].

2.7.3 Exploration–Exploitation in FL Aggregation

Recent research has begun to treat the inclusion of clients in FL as an exploration–exploitation problem, leveraging multi-armed bandits to learn which participants are the most reliable over time. FedSBS, for example, employs an ϵ -greedy policy where clients are scored based on their contribution to global model improvement. While this approach avoids over-committing to early high-scoring clients by occasionally exploring others, its scoring is primarily based on local accuracy or data volume rather than more nuanced similarity measures between updates [27].

FedRTS treats the server-side pruning as a multi-armed bandit: every weight is an arm, and in each outer communication round the server samples from a Beta posterior for every arm and activates the K arms with the highest Thompson-Sampling draws to form the next sparse mask. After clients train locally, the server updates those posteriors with a fused reward signal that blends aggregated model statistics with the top-gradient indices uploaded by participating clients, so the aggregation decision continually balances

exploration of new connections and exploitation of historically “high-reward” ones rather than relying on deterministic magnitude thresholds [28]. In contrast to these methods, this thesis proposes an adaptive aggregation strategy that builds on TrMean and uses an epsilon greedy exploration vs. exploitation, utilizing the TrMean aggregation rule.

Chapter 3

Method

The Method chapter details the practical pathway from concept to evidence. It first outlines the iterative research process that turned the high-level research questions into concrete design choices, spanning literature review, simulator architecture, attack implementation, and defense integration. Next it describes the multi-node Kubernetes testbed, the data-partition schemes, and the measurement pipeline that captures both model-level metrics (accuracy, loss, convergence) and aggregator-level metrics (late benign inclusion rate and mean malicious exclusion rate). Finally, the chapter explains how each experiment is parameterized, executed, and logged, providing the reproducible foundation on which the subsequent Results and Discussion chapters are built.

3.1 Research Process

The work undertaken in this thesis follows an iterative approach with theoretical exploration, systematic system-building and empirical testing. Below is an overview of how each major step connects to the research questions and goals and how early insights informed the subsequent steps while all advancing toward the final results.

1. Initial Project Planning and Goal Setting

Early on, we clarified the project scope and formulated the main objectives: Transitioning from a single-node to a multi-node FL simulator, Implementing a suite of robust aggregation strategies (Multi-KRUM, Trimmed Mean, Divide-and-Conquer) and eventually designing, implementing and testing the novel adaptive aggregator. We identified constraints such as computational resources, dataset

availability and time frames and then established a high-level metrics—exclusion/inclusion rates of malicious and benign clients and global model accuracy under adversarial conditions.

2. Literature Review and Research Gap Identification

We conducted a comprehensive review of FL, focusing on data and model poisoning attacks, robust aggregation methods and dynamic client-participation scenarios. This review shed light on shortcomings in existing solutions, particularly their reliance on static assumptions (IID data, static client sets, naive attacks) and it also confirmed the need for a multi-node simulator with realistic attack models, as well as a more adaptive, exploration-exploitation-based aggregator to handle late-joining or heterogeneous clients.

3. Refined Plan and Infrastructure Decisions

Building on insights from the literature, we finalized decisions regarding our multi-node architecture, selecting Kubernetes for orchestrating containerized FL clients in a distributed environment. We also defined core data partition strategies (IID vs. non-IID class imbalanced, volume balanced vs. imbalanced) to test aggregator performance more rigorously. This step makes sure that the experiments would capture more realistic, variable conditions which are frequent in a distributed FL setting.

4. Initial System Integration

We updated the code to be compatible with the latest FEDn, our chosen FL framework. During this phase we verified container communication, dataset partitioning, multi-node orchestration and communication with FEDn Studio.

5. Multi Node Architecture

In this phase, we scaled the simulator from a single node setup now utilizing FEDn Studio for the aggregation and round policy communication, to a multi-node architecture capable of launching multiple clients across different VMs through utilizing Kubernetes.

6. Inclusion of Existing Aggregation Rules

Next, we integrated known robust aggregators (Multi-KRUM, Trimmed Mean, Divide-and-Conquer) into the simulator.

7. Client-Side Design and Attack Implementation

In the following step we developed a condition to let clients participate

in the training of the global model at a later stage of the training rounds, reflecting the real-world problem of intermittent connectivity. As well as created some predefined data partitions for a set of tests to be executed next.

8. Comprehensive Experimental Analysis

After the attacks and defenses were deemed stable, we design structured experiments varying parameters such as:

- Label distribution and volume distribution
- Client Arrival: Fraction of malicious vs. honest clients and timing of arrival.
- Attack Types: Data poisoning attack, label flipping and the model poisoning attack LIE for MNIST-PyTorch model.

We record metrics such as global model accuracy, late benign inclusion rate (LBIR) and mean malicious exclusion rate (MMER) and finally test loss area under curve (AUC) to evaluate each aggregator's effectiveness.

9. Design and Implementation of a Novel Aggregator

Observations from the prior experimental phases (e.g., excessive exclusion of honest late-joiners) motivate the development of a new, adaptive aggregator strategy. Using the exploration-exploitation principles, we aim to better accommodate dynamic client participation while maintaining strong mitigation against malicious activity.

10. Benchmarking and Comparative Evaluation

We then compare this novel aggregator against existing robust aggregation rules under identical settings. By utilizing the same experimental setup (ie. dataset balance, distribution skew, client arrival and adversarial tactics) as the existing aggregation rules were test upon. We measure whether our approach reduce honest exclusion rates or improve global model accuracy without compromising defensive capabilities.

11. Conclusion and Recommendations

Finally, we synthesize key findings, reflect on how well our multi-node simulator satisfies its goals and how effectively each robust aggregation strategy perform in dynamic, adversarial FL settings.

3.2 Data Collection

This thesis uses a purely simulation-based approach to investigate adversarial behavior and robust aggregation methods in FL. The sections below clarify how these experiments were sampled and how many experiments are run.

3.2.1 Evaluation Scenarios

All results in this thesis is generated through controlled runs of the Kubernetes-based multi-node simulator. To make the experimental space explicit and reproducible, we describe below (i) the dimensions that are varied, (ii) the public datasets used, (iii) the resulting experiment matrix and (iv) how these choices map onto practical FL deployments.

3.2.1.1 Scenario Dimensions

Each simulation condition is defined by the following factors:

- Total clients: $N = 10$.
- Malicious fraction: $p_{\text{adv}} \in \{0.10, 0.20, 0.30\} \Rightarrow \{1, 2, 3\}$ malicious clients respectively.
- Late joining policies:
 - (a) Malicious: exactly $p_{\text{adv}} N$ malicious clients join at round 5, the remaining clients (benign) start at round 0.
 - (b) Benign: the same number of benign clients ($p_{\text{adv}} N$) join at round 5, the remaining clients (benign and malicious) start at round 0.

Thus, each malicious proportion is tested twice, once with late malicious and once with late benign clients, keeping every other experimental parameter identical.

- Data partitioning: IID and non-IID label distribution, with varying degrees of volume-distributions, balanced, partially imbalanced and imbalanced data partitions.
- Aggregation rule (AGR): FedAvg, Multi-KRUM, Trimmed Mean, Divide-and-Conquer and the proposed Adaptive AGR.

3.2.1.2 Public Dataset

- MNIST (70 000 images, 10 classes): we apply Dirichlet splits to obtain IID and non-IID label distributed and volume distributed data partitions.

3.2.1.3 Experiment Matrix

Combining the factors above yields 3 (malicious fraction) $\times 2$ (late-joining policy) $\times 4$ (label and volume distribution) $\times 5$ (AGRs) $+ 3$ (malicious fraction) $\times 2$ (late-joining policy) $\times 2$ (label and volume distribution) $\times 5$ (AGRs) = 180 unique conditions.

3.2.1.4 Practical Justification

- Dynamic participation and a 10%, 20%, 30% adversary rate reflects realistic degrees of attack configurations [4].
- The two attacks (LF, LIE) spans data poisoning and model poisoning attacks, cited in [7, 10].
- Different label and volume distributions probe how AGRs cope when some classes are scarce and data is not evenly distributed, a frequent reality in federated learning [4].

3.3 Multi-Node Simulator Architecture

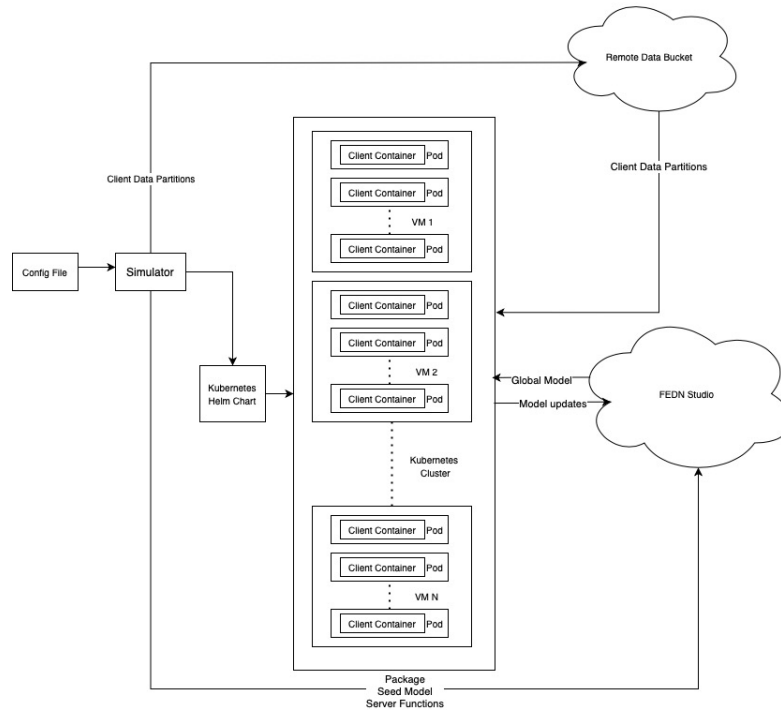


Figure 3.1: An overview of the simulation architecture.

3.3.1 Multi-Node Simulator Overview

The multi-node simulator emulates a realistic federated-learning deployment in which many independent clients, benign or adversarial, train locally and periodically transmit their model parameters to a central aggregator.

At the core of the simulator we use FEDn Studio, Scaleout Systems’ managed hosting service, to run the aggregation tier and round handling. Studio’s recently introduced server-side functions API lets us inject custom aggregation logic, an essential requirement for evaluating mitigation strategies against malicious behavior. By off-loading the server components to Studio, the local artifacts can focus exclusively on client-side functionality.

Each client is packaged as a Docker image and scheduled as a Kubernetes Deployment. Kubernetes automatically places pods on available virtual machines, restarts them on failure, and scales the population up or down. Helm provides parameterised charts, so a single command can launch an experiment with any mix of benign and malicious clients.

The entrypoints `train.py`, `validate.py`, and the supporting data-handling routines were refactored to authenticate against Studio, push local model updates, and retrieve the latest global model. A config-file is set up for the user to enter the number of clients, which attack and mitigation strategies to use for the simulation, hyper-parameters and the distribution of the data partitions for the clients. Because the containers are immutable, no recompilation is required between runs, new experiments can start in seconds.

The simulator presented in [15] hosted the entire FEDn stack on two local virtual machines—one for the aggregator and one for a single client. The new multi-node design removes that infrastructure burden, supports an arbitrary number of clients, and exposes the aggregation layer through server-side functions. Apart from this re-architecting, the Docker-based workflow and the attack plug-in interface from [15] are preserved.

3.3.2 Containerization and Helm Chart Design

A Docker-based image was constructed to encapsulate each client’s tasks, including retrieving its designated data partition, performing localized training (potentially with adversarial modifications) and transmitting updates for global aggregation. This container can be considered “stateless” insofar as it only holds executable code and dependencies, while data resides at a remote data bucket. The container’s interface is standardized via environment variables (e.g., client ID, attack type or gradient inflation factor), which allows for multiple instances of the same image to mimic different client behaviors in a distributed scenario.

The Helm chart created defines how these client deployments are managed and specifies the clients environment variables for each pod. The file `chart/values.yaml` provides a skeleton for, among other aspects, the Docker image location, aggregator endpoint and user credentials. The format from `chart/values.yaml` are then copied at runtime by `test_simulation.py` that generates a new file `values-temp.yaml` with config files parameters (e.g., the number of benign and malicious clients or the aggregator’s IP). Subsequently, the chart template `client-env.yaml` uses these parameters to spin up Kubernetes deployments, assigning unique environment variables to each client pod.

3.3.3 Data Partition and Distribution

The single-node simulator expected users to host the dataset locally and then split the dataset into N client partitions, which were copied into the client containers. To improve on this, users can now either use a remote data bucket where the client partitions are stored prior to initializing the Kubernetes cluster or provide the simulator with the host and access keys for another remote data bucket where it can download pre-partitioned data splits for the clients. This allows a more pluggable architecture, where users can experiment with different non-IID data partitioning methods and then upload them to a data bucket and save for future experiments. At runtime the clients then access and download their data partitions.

3.3.3.1 IID label and volume data partitions

For an independent-and-identically-distributed (IID) label split, every client receives data drawn with the same class proportions as the full dataset.

- **Balanced (`splitset`):** we assign exactly $n = \lfloor |\mathcal{D}|/N \rfloor$ samples to each of the N clients, preserving the global label histogram.
- **Imbalanced (`iid_unbalanced_split`):** client sizes are first drawn from a Dirichlet distribution $\text{Dir}(\mathbf{1})$ and then rescaled to the dataset size, subject to a minimum quota of 10% of the average client size to avoid very small partitions. Within every client we keep the same label ratios as the full dataset.

In both cases every client's label distribution remain IID with respect to the underlying distribution, even though the total sample counts may differ.

3.3.3.2 Non-IID client partitions

To emulate statistical heterogeneity we follow the common Dirichlet-label-skew recipe:

- For each client i we draw a class-probability vector $\pi_i \sim \text{Dir}(\alpha \mathbf{1})$ with $\alpha = 0.5$, making some classes dominant and others scarce.
- We then allocate samples according to π_i , either
 1. **Partially Imbalanced:** each client receives the almost the same number of samples
 2. **Imbalanced:** client sizes are again taken from a Dirichlet draw and limited to at least 10% of the average size.

3.3.4 Client Lifecycle and Attack Execution

When a client pod starts, it retrieves and instantiates the Docker container, loading the environment variables that specify operational parameters such as aggregator IP or FEDn Studio authentication token. Once it successfully connects to FEDn Studio, each client stays idles waiting until it receives further instructions from its assigned combiner.

Once FEDn Studio initiates a new “round”, ie. a training cycle, the clients receive a training instruction alongside a seed model and any relevant package files (e.g., scripts for data handling, training and validation). Each client then reads its environment variables, fetches its designated partition from the remote data bucket and begins the local training process. During this step, the client reads its environment variables to check whether it is malicious; if so, it applies the corresponding adversarial strategy, such as label flipping, gradient boosting, backdoor insertion or little-is-enough.

After training completes, the client transmits its updated parameters back to the combiner, which merges them with other clients’ updates into a partial model. This partial model is forwarded to the reducer, forming a globally aggregated model. The updated global model is then returned to the clients, prompting them to validate training and validation loss and accuracy on their respective local and updated global models. The resulting metrics are reported back to FEDn Studio, where a graphical interface displays the performance and effectiveness of any adversarial or defensive strategies currently in place. This process repeats for a user-defined number of rounds, as configured within FEDn Studio.

The effectiveness of both attacks and mitigation strategies is assessed through a combination of model-level and aggregator-level metrics, collected at each communication round $t \in \{1, \dots, T\}$ of training.

Global Model Accuracy and Loss: Let $\mathcal{D}_{\text{test}}$ and $\mathcal{D}_{\text{train}}$ denote the test and training dataset partitions, respectively. After each round, the updated global model w_t is evaluated on both datasets:

$$\text{Accuracy}_{\text{test}}(t) = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x_i, y_i) \in \mathcal{D}_{\text{test}}} 1[\arg \max f(w_t; x_i) = y_i],$$

$$\text{Loss}_{\text{test}}(t) = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x_i, y_i) \in \mathcal{D}_{\text{test}}} \ell(f(w_t; x_i), y_i),$$

Using the method above, we present the test final accuracy as: $\text{Accuracy}_{\text{test}}(T)$.

Finally using the point-wise loss, we present the whole trajectory $\{\text{Loss}_{\text{test}}(t)\}_{t=\tau}^T$ area under the curve. Using the trapezoidal rule,

$$\text{AUC}_{\text{loss, test}} = \sum_{t=1}^{T-1} \frac{\text{Loss}_{\text{test}}(t) + \text{Loss}_{\text{test}}(t+1)}{2}.$$

Client Inclusion and Exclusion Metrics:

Fix a window of rounds $\{\tau, \dots, T\}$ and denote by $\mathcal{C}_t^{\text{sel}} \subseteq \mathcal{C}$ the clients whose updates are accepted in round t .

Assume that every client in the subset $\mathcal{S} \subseteq \mathcal{C}$ first appears at the same round $\tau_{\text{late}} \geq \tau$. (The set $\mathcal{S}$ can be the late-malicious clients $\mathcal{M}_{\text{late}}$ or the late-benign clients $\mathcal{B}_{\text{late}}$.)

$$N_{\text{opp}}(\mathcal{S}) = (T - \tau_{\text{late}} + 1) |\mathcal{S}|, \quad (\text{total opportunities})$$

$$N_{\text{sel}}(\mathcal{S}) = \sum_{t=\tau}^T |\mathcal{C}_t^{\text{sel}} \cap \mathcal{S}|, \quad (\text{actual selections})$$

$$\text{IR}(\mathcal{S}) = \frac{N_{\text{sel}}(\mathcal{S})}{N_{\text{opp}}(\mathcal{S})} \in [0, 1], \quad (\text{inclusion rate})$$

$$\text{ER}(\mathcal{S}) = 1 - \text{IR}(\mathcal{S}). \quad (\text{exclusion rate})$$

$$\text{MMER} = \text{ER}(\mathcal{M}_{\text{late}}), \quad \text{LBIR} = \text{IR}(\mathcal{B}_{\text{late}}).$$

Defense Robustness: A strong mitigation strategy should yield high MMER while maintaining a high LBIR. These two quantities are reported alongside the global model accuracy to identify trade-offs between strictness and vulnerability.

Convergence and Stability: Finally, we analyze the trajectory of the loss and accuracy curves to evaluate training stability. Oscillations, plateaus, or sharp drops may signal instability induced by adversarial updates or excessive exclusion of diverse (but benign) updates.

All metrics are logged per round and stored in structured JSON format.

3.3.4.1 Label Flipping

Label flipping in [15] follows a static strategy, flipping labels between class pairs as in [4]. The strategy utilized is $L - l - 1$, with L representing the total number of classes. Resulting in an inversion of the labels with a shift of 1 to the left.

Algorithmically:

1. Load client training data (X_{train}, y_{train}).
2. Apply label flipping strategy.
3. Load global model parameters.
4. Complete training.
5. Send updated parameters to the server.

3.3.4.2 Gradient Boosting

The gradient boosting in [15] extends the method from [7] by negating the gradient and amplifying parameter updates by factors of 2, 10 and 100.

Algorithmically:

1. Load client training data.
2. Load global model parameters into the local model.
3. Boost parameters by factor λ .
4. Complete training.
5. Send updated parameters to the server.

3.3.4.3 Little-Is-Enough

The Little-is-Enough attack was implemented in the single node simulator. Since no defense mechanisms were implemented in the single node simulator, [15] introduced a variant of the LIE attack. The adversary, aware of the number and proportion of benign versus malicious clients, aims to redirect global model parameters in their chosen direction.

Initially, the attacker identifies the global parameter trajectory through multiple updates. Once identified, the malicious clients actively steer the parameters opposite to the benign direction, carefully formulating updates to overpower benign clients. The malicious clients utilize historical data on global model updates to optimize their influence.

Algorithmically:

1. Load the global model.

2. Initiate attack at intervals (e.g., every third round).
3. Compute benign client proportion (λ) and set the pull factor (p).
4. Retrieve global models G_t (latest), G_{t-2} (two rounds earlier) and their parameter P_t and P_{t-2} respectively.
5. Compute malicious update parameters: $P_{t+1} = P_{t-2} - p \cdot \lambda \cdot (P_t - P_{t-2})$.
6. Send malicious updates for aggregation.

3.3.5 Mitigation Strategies

With server-side functions in FEDn, a user of FEDn can easily implement a desired mitigation strategy and upload it to the FEDn server which prompts the combiner and reducer to adopt the chosen aggregation policy for the session. The user simply selects its chosen mitigation strategy in the config file. This approach aligns with a scalable and pluggable architecture, where a user can define a new mitigation strategy, push it to the server and apply it to the simulator with minimal effort.

The implementation of each of the mitigation strategies applied will be discussed below together with the pseudo-algorithms of each.

3.3.5.1 Trimmed Mean (TrMean)

In the three-tier architecture of FEDn the aggregation step occurs twice. First, each combiner aggregates the updates from its assigned clients to produce a partial model update. Subsequently, the reducer aggregates all partial updates into a final global model update.

Trimmed Mean (TrMean) filters outliers by examining each coordinate (or parameter) across all client updates. Let $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ be N local updates and let $\beta \in (0, 0.5)$ denote the fraction to trim at each tail. If $N < 3$, the implementation falls back on FedAvg aggregation. Otherwise, for each dimension j , we sort all values $\{x_i^{(j)}\}_{i=1}^N$ in ascending order, remove the lowest $\lfloor \beta N \rfloor$ and highest $\lfloor \beta N \rfloor$ entries and then compute the FedAvg on the remaining entries. This process is repeated for every coordinate to obtain the final aggregated update.

Listing 3.1: Trimmed Mean (TrMean) aggregator pseudocode

```
function TRIMMED_MEAN(X[1..N],  $\beta$ ):
    if  $N < 3$ :
```

```

    return FED_AVG(X)
  for each dimension j in the model:
    vals ← j-th coordinate from X[i] for i=1..N
    sort(vals)
    lower ← floor( $\beta$  × N)
    upper ← N - lower
    trimmed_vals ← vals[lower : upper]
    aggregated[j] ← FedAvg(trimmed_vals)
  return aggregated

```

3.3.5.2 KRUM

As in the Trimmed Mean approach, aggregation in FEDn is performed in two stages due to its three-tier architecture. Each client is connected to a combiner and each combiner aggregates the updates from its assigned clients to form a partial model update. These partial updates are then forwarded to the reducer, which computes a global model update by aggregating all partial updates into a global model.

KRUM seeks a single “most central” update, particularly valuable when a fraction of clients might be malicious. Suppose there are N updates $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and a tolerance parameter f for the maximum number of adversarial clients. For each update $\mathbf{x}_i$, we compute Euclidean distances to every other $\mathbf{x}_j$, sort those distances and sum the smallest $(N - f - 1)$ values to obtain a “KRUM score” S_i . The update that yields the lowest score S_i is returned as the aggregated result. In this thesis we use $f=1$.

Listing 3.2: KRUM aggregator pseudocode

```

function KRUM(X[1..N], f):
  if N < 2:
    return X[1]  # if only one update
  for i in 1..N:
    Distances ← || X[i] - X[j] || for j ≠ i
    sort(Distances)
    neighbors ← Distances[1 .. (N - f - 1)]
    S_i ← sum(neighbors)
  k ← argmin_i S_i
  return X[k]

```

3.3.5.3 Multi-KRUM

Similarly to the previous AGRs mentioned the aggregation in FEDn is performed in two stages due to its three-tier architecture. Each client is connected to a combiner and each combiner aggregates the updates from its assigned clients to form a partial model update. These partial updates are then forwarded to the reducer, which computes a global model update by aggregating all partial updates into a global model.

Multi-KRUM extends the basic KRUM method by selecting multiple “central” updates denoted by k than just one. After computing the KRUM score for each client update (i.e., summing its distances to its closest $N - f - 1$ neighbors), Multi-KRUM identifies the k updates with the lowest scores. Those k updates are then aggregated using FedAvg to produce the global update. In this thesis we use $f=1$, and $k=3$, throughout all the experiments using Multi-KRUM.

Listing 3.3: Multi-KRUM aggregator (pseudocode)

```
function MULTI_KRUM(X[1..N], f, k):
    for i in 1..N:
        Distances  $\leftarrow$   $\|X[i] - X[j]\|$  for  $j \neq i$ 
        sort(Distances)
        neighbors  $\leftarrow$  Distances[1 .. (N - f - 1)]
        S_i  $\leftarrow$  sum(neighbors)

    # Sort all updates by KRUM score S_i
    sorted_by_score  $\leftarrow$  sort(X, key=S_i)
    # Take the top k "most central" updates
    selected  $\leftarrow$  sorted_by_score[1..k]

    return FedAvg(selected) # for k select updates
```

3.3.5.4 Divide-and-Conquer (DnC)

Similarly to the previous AGRs mentioned the aggregation in FEDn is performed in two stages due to its three-tier architecture. Each client is connected to a combiner and each combiner aggregates the updates from its assigned clients to form a partial model update. These partial updates are then forwarded to the reducer, which computes a global model update by aggregating all partial updates into a global model.

In each round every combiner receives N flattened client updates $\{\nabla_1, \dots, \nabla_N\} \subset R^d$ and applies the following five-step procedure before forwarding its partial model to the reducer.

First, it draws a random coordinate subset $\mathcal{I} \subset \{1, \dots, d\}$ of size $b = \lceil p d \rceil$; throughout this thesis we fix $p = 0.08$, which was used on the MNIST dataset in [19]. The restricted, mean-centred updates $\nabla_i^{(\mathcal{I})} - \bar{\nabla}$ are stacked row-wise into a matrix $\mathbf{Z} \in R^{N \times b}$. A single singular-value decomposition of $\mathbf{Z}$ yields the leading right-singular vector u , which exposes the direction along which clients differ the most. Each client receives a spectral outlier score $s_i = |(\nabla_i^{(\mathcal{I})} - \bar{\nabla})^\top u|$; the combiner discards at most f updates with the largest scores ($f = 1$ in all our experiments). The sub-sampling, centering and pruning routine is repeated R times with fresh coordinate draws ($R = 1$ in this thesis). Only clients that survive every repetition are considered trusted and their updates are combined with FedAvg.

Listing 3.4: DnC combiner (parameterised by p, f, R)

```
function DNC_AGGREGATE(updates [1..N], p, f, R):
    survivors ← {1..N}
    for r in 1 .. R:
        I ← random_coordinates ( p·d )           # sub-sample
        Z ← stack_rows (updates [i][I])          # N × b
        Z ← Z - column_mean(Z)                   # centre
        u ← top_right_singular_vector(Z)          # SVD
        scores[i] ← | Z[i] · u |
        keep ← indices_of_lowest(scores, N - f)
        survivors ← survivors ∪ keep
    if survivors = ∅:
        return FedAvg(updates)                   # fallback
    return FedAvg(updates[i] for i in survivors)
```

3.4 Adaptive Mitigation Strategy

Robust aggregation rules such as the coordinate-wise Trimmed-Mean offer strong Byzantine resilience, yet they treat every communication round identically and therefore may (i) suppress informative but infrequent updates from honest clients and (ii) still allow poisoned portions of parameters to slip through on the subset of coordinates if an adversary remains inside the non-trimmed region.

3.4.1 Exploration–Exploitation TrMean (EE-TrMean)

To mitigate the shortcomings of a static Trimmed Mean, we couple it with an ϵ -greedy exploration policy. In every communication round t the server receives a set $\mathcal{C}_t$ of $N = |\mathcal{C}_t|$ client updates. Each client index $i \in \mathcal{C}_t$ maintains participation and trust counters, rounds_i and score_i , both initialised to zero; upon participation we set $\text{rounds}_i \leftarrow \text{rounds}_i + 1$. If at least one newcomer appears (i.e. $\text{rounds}_i = 1$ for some i), we force a fully exploratory round by setting $\varepsilon_t = 1$, otherwise $\varepsilon_t = \varepsilon$. With probability ε_t we aggregate all clients, $S_t = \mathcal{C}_t$ (exploration). When exploration is not chosen we build a trusted subset

$$S_t = \left\{ i \in \mathcal{C}_t \mid \bar{s}_i \geq \alpha \max_{j \in \mathcal{C}_t} \bar{s}_j \right\}, \quad \bar{s}_i = \frac{\text{score}_i}{\max(1, \text{rounds}_i)},$$

and, if fewer than three clients exist we revert to $S_t = \mathcal{C}_t$. The server then performs the coordinate-wise Trimmed Mean on $\{x_i^{(j)} : i \in S_t\}$, it sorts the values, discards the lowest and highest elements and averages the remaining block with FedAvg. Clients that survive trimming for a given coordinate are rewarded via $\text{score}_i \leftarrow \text{score}_i + 1$. The exploration parameter decays according to $\varepsilon \leftarrow \max(\varepsilon_{\min}, \gamma\varepsilon)$, while the trust-threshold is ramped within each L -round session by

$$\alpha \leftarrow \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \sqrt{\frac{t \bmod L}{L - 1}}.$$

In this thesis we use $\beta = 0.1$, $\alpha_{\min} = 0.60$, $\alpha_{\max} = 0.76$, $L = 30$, $\varepsilon_{\min} = 0.05$, and $\gamma = 0.87$.

Listing 3.5: EE-Trimmed-Mean (EE-TrMean) with epsilon-decay and alpha-ramp

```
function EE_TRMEAN(X[1..N], beta,
                  alpha_min, alpha_max,
                  L,      # session length
                  epsilon, epsilon_min, gamma,
                  newcomer_flag,
                  t):

    # alpha-ramp in an L-round session
    prog ← (t mod L) / (L - 1)
    alpha_t ← alpha_min + (alpha_max - alpha_min) * sqrt(prog)
```



```

# exploration rate for this round
epsilon_round ← 1 if newcomer_flag else epsilon

# -- exploration vs. exploitation
if rand() < epsilon_round:
    S ← {1, ..., N}                                # explore: use all clients
else:
    leader ← max_i score[i] / rounds[i]
    S ← { i | score[i]/rounds[i] ≥ alpha_t · leader }
    if |S| < 3:                                       # safety floor
        S ← top-3 clients by score or {1, ..., N}

# -- coordinate-wise Trimmed Mean on S
for each dimension j in the model:
    vals ← [ X[i][j] for i in S ]
    sort(vals)
    lower ← floor(beta × |S|)
    upper ← |S| - lower
    trimmed_vals ← vals[lower : upper]
    aggregated[j] ← FedAvg(trimmed_vals)

# update scores for surviving clients
for each i ∈ S \ {clients trimmed away}:
    score[i] ← score[i] + 1

# epsilon decay (unless locked at epsilon_min)
epsilon ← max(epsilon_min, gamma · epsilon)
return aggregated

```

EE-TrMean is built upon the idea that the classical β -Trimmed-Mean already produces a useful verdict for every client: if a client's update survives trimming it is likely honest, whereas systematic trimming hints at malicious behaviour. EE-TrMean converts this verdict into a lightweight reward signal and keeps, for each client, an exploitation score. The score averaging prevents honest-but-different clients from being dropped prematurely because their first updates looked unusual in a highly heterogeneous (non-IID) scenario.

The alpha ramp showed to improve on the epsilon greedy algorithm encouraging diversity in the initial rounds, while in later rounds increasing the threshold to exploit clients that have proved consistently reliable.

3.5 Experimental Setup

The following section will highlight the hardware and software tools used in the implementation, evaluation and validation of the proposed methods.

Hardware:

- 6 Virtual Machines (VMs), each equipped with 8 GB of RAM and 4 CPUs, for simulating a distributed federated learning environment.
- Standard personal computer for development, testing and documentation purposes.

Software:

- FEDn as the federated learning framework for decentralized training and experimentation.
- Scikit-learn (Sklearn) for implementing logistic regression model.
- Docker for containerization.
- Kubernetes and Helm for deploying and managing multi-node federated learning workloads.
- Pytorch for implementing the convolutional network model.
- Python along with essential libraries (e.g., NumPy, Pandas, Matplotlib) for data handling and visualization.
- Git and GitHub for version control.

3.6 Evaluation framework

The evaluation framework is designed to unify the data collection, analysis and comparison processes in a systematic manner:

1. Experimental Protocol: Each experiment follows a standard protocol detailing the number of clients, distribution of data, ratio of malicious vs. benign clients, aggregator settings and run duration in terms of FL rounds.

2. **Baseline vs. Defense Comparisons:** We benchmark each scenario against a naive FedAvg aggregator (no defense) to quantify the incremental benefit of robust AGRs like Trimmed Mean, DnC and Multi-KRUM.
3. **Performance Indicators:** We track final model test accuracy, test loss area under curve (AUC), Mean Malicious Exclusion Rate (MMER) and Late Benign Inclusion Rate (LBIR). Together, these metrics highlight the trade-offs between security and efficiency.

Chapter 4

Results

The results chapter starts by affirming the goals reached through the construction of the Multi-Node simulator. It then follows with the series of experiments performed in this thesis. The experiments have been divided up into 6 sections, where all experiments for each attack and partitioning scheme are listed in the same table. We present final test accuracy, AUC, MMER and LBIR for each experiment. Attached in the appendix, is a subset of the experiments, that were deemed interesting to visualize, each accompanied by its respective test-accuracy plot and a participation timeline that visualize every client's participation.

4.1 Multi-Node FL Simulator

While the Method Chapter explained how the multi-node simulator was implemented, the purpose of this section is to document the concrete artifacts produced and, in doing so, answer Research Question 1: How can a scalable, modular and pluggable multi-node FL simulator be implemented so that it supports realistic emulation, controlled attack injection and seamless defense integration?

Single Entrypoint Experiment parameters reside in one declarative file– `config.yaml`. In which a user defines: attack type, mitigation strategy, number of malicious clients, number of benign clients, late-join offset and total rounds. `config.yaml` is then read by the launcher script, `test_simulation.py`, which sets environment variables for the client pods, that are then accessed and used during the different phases of the session. The keys in `config.yaml` propagate automatically throughout the simulator, without the need to create a new image.

Scalability To verify that the Kubernetes deployment scales predictably, we measured peak CPU and memory consumption while increasing the client count from 5 to 30. The CPU and memory usage grow near-linearly (see Figures 28 and 29 in the appendix), illustrating that each additional client imposes a near constant resource cost on the host infrastructure, thus making it easy and achievable to calculate the needed resources for a specific client count.

Stateless client pods Each client runs as a Kubernetes pod. No local state is kept between rounds, so if a pod is evicted or crashes, the federation proceeds, resulting in a fault-tolerant simulation.

Kubernetes + Helm Experiment parameters are first declared in the simulator configuration file, `config.yaml`. At launch, `test_simulation.py`:

1. Reads the file,
2. Copies a template `values.yaml`,
3. Writes a run-specific `values-temp.yaml` and `client-env.yaml` that separates global settings from per-client environment-variables
4. Calls `helm install -f values-temp.yaml sim`.

The script then iterates over the client list defined in `values-temp.yaml` and creates one pod per client, each receiving its own parameter set, e.g. malicious clients receives a malicious flag while benign doesn't. Consequently, scaling from ten to hundreds of clients is easily done by changing one value in `config.yaml`.

Mitigation All mitigation strategies are coded in `server_functions.py`, a subclass of `ServerFunctionsBase` that FEDn calls via three hooks: `client_selection()`, `client_settings()` and `aggregate()`, which forbids arbitrary imports. Because sending information to `server_functions.py`, once uploaded cannot be done, the desired parameters are instead fetched from the client name:

1. `docker_client.py` receives environment variables from Helm (`MITIGATION=ee-trmean, LATE=5, ...`)
2. Encodes them into the client name
3. The client name is then available for `server_functions.py` through the model updates it receives.

4. Inside `server_functions.py` our subclass of `ServerFunctionsBase` parses that string (`_quick_parse()`) to decide which internal aggregator method to call, which client indices to exclude, etc.

`server_functions.py` currently holds 5 aggregation methods: FedAvg, TrMean, Multi-KRUM, DnC, EE-TrMean, each implemented as a private method, such as `_aggregate_multi_krum()`.

Adding a new mitigation therefore only boils down to adding a aggregation method to `server_functions.py`. The controller and combiner receives `server_functions.py` from the `client.start_session` command in `test_simulation.py`. So no image rebuild is needed and the simulator's pluggability property is preserved.

Modular attacks The combiners sends calls to `docker_client.py`, for training or validation. When the client receives a train call, the client then forwards that request to `train.py`. `train.py` that then reads the experiment desired parameters from environment variables, which can trigger malicious behavior. Added is `attack.py`, where users can implement additional attacks to be used in the simulator. Implemented are: label-flip, LIE, back-door attacks, gradient-boosting.

Remote partitions Before training, each pod downloads its data partition from a remote data storage. Because data lives external to the image, swapping in MNIST → FEMNIST or custom splits involves only uploading new data partitions and updating `config.yaml`, addressing pluggability.

Realistic FL Dirichlet (α) samplers generate IID/non-IID label distributed and balanced, partially imbalanced and imbalanced splits for the MNIST dataset, late-joining policy is enacted in `server_functions.py`, which fetches the desired parameters from the client names, which in turn is fetched from environment variables of the pod, from `config.py`. Together, these features emulate real-world FL deployments.

Through these decisions above, the simulator (i) scales horizontally across VMs, (ii) supports adding new attacks and defenses with minimal effort and (iii) faithfully emulates data heterogeneity and late-joining clients. Hence, the implementation meets all three design criteria posed in RQ 1.

4.2 Experiments

This section covers the testing of five robust aggregation mechanisms, TrMean, Multi-KRUM, Divide-and-Conquer, EE-TrMean and the FedAvg baseline, tested on a comprehensive matrix of adversarial settings, using the multi-node FL simulator. For each combination of attack type (label-flipping or LIE), label distribution (IID vs. non-IID) and volume-balance (balanced, partially imbalanced and imbalanced), late-joining policy and malicious-client fraction, we report both model-level metrics (final test accuracy, AUC) and inclusion and exclusion metrics, MMER and LBIR, that show how many of the late honest and malicious updates are actually included in, or excluded from, the global model. This dual focus lets us quantify the classic security–utility trade-off: a defense is only robust if it suppresses poisoned updates without sidelining valuable contributions from benign but atypical clients. The following subsections therefore present twelve result tables (Tables 4.1 – 4.12) that trace each strategy’s behavior across all experimental dimensions, providing the empirical basis for the discussion and conclusions that follow.

This section highlights the results related to:

- **Research Question 2:** To what extent do contemporary robust aggregation rules, such as Multi-KRUM, Trimmed Mean and Divide-and-Conquer, successfully discriminate between honest and malicious clients when the federation experiences non-IID data distributions and dynamic client participation?
- **Research Question 3:** How can we develop and validate an adaptive robust aggregation strategy that dynamically adjusts to variations in client reliability, data heterogeneity, and participation timing, while preserving resilience against adversarial attacks and maintaining global-model performance?

Prior to moving to the results tables, there are a few important remarks to be made. TrMean was not able to fully filter out any malicious client in any of the 180 experiments. This is a result of how the aggregation works. TrMean performs a coordinate-wise filtering, which means that for this model, we have over 50 000 coordinates. Furthermore since we only count a client excluded if all of the coordinates were filtered out, during one round of the aggregation, the probability that all of a clients parameters were filtered out, is quite low. Another reason lies in the soft setting of the clipping factor of TrMean in this thesis. The clipping factor was set to clip only the top 10%

and bottom 10% of the coordinates. Meaning that in cases such as 30% malicious client activity, it could not possibly filter out all of the malicious client updates. A higher clipping factor could have resulted in TrMean fully filtering out some malicious clients, but with the sheer number of coordinates even that likelihood is low.

Finally, training was limited to 25 communication rounds, with one local epoch per round, so the results cannot fully reveal the long-term behavior of the AGRs or the sustained effectiveness of the attacks. Two 80 round experiments illustrating the long term behavior of the mitigation strategies and the attacks can be found in the appendix, one for LIE and one for Label Flipping.

4.2.1 Label flipping - IID Balanced

Mitigation	Late Policy Malicious Activity	Accuracy			AUC		
		10 %	20 %	30 %	10 %	20 %	30 %
FedAvg	Benign	89.9 %	90.1 %	73.5 %	19.072	18.380	35.174
	Malicious	90.6 %	85.6 %	74.8 %	18.626	22.231	29.751
TrMean	Benign	90.9 %	88.3 %	78.3 %	17.568	20.909	31.636
	Malicious	90.8 %	88.4 %	79.0 %	17.504	19.912	26.774
Multi-KRUM	Benign	90.8 %	90.9 %	90.9 %	17.282	17.291	17.088
	Malicious	90.6 %	90.9 %	84.8 %	17.292	17.320	26.723
DNC	Benign	91.0 %	89.5 %	83.6 %	17.291	19.539	26.624
	Malicious	91.0 %	89.6 %	83.9 %	17.328	18.915	23.520
EE-TrMean	Benign	91.0 %	90.8 %	78.3 %	17.387	19.079	31.635
	Malicious	91.0 %	90.8 %	79.0 %	17.381	18.377	26.775

Table 4.1: Label-flipping – IID – Balanced: Test Accuracy and Test Loss AUC, computed over all participating clients. Each AGR results is shown in two rows, corresponding to the two late-policies. Columns separate the results at the three malicious proportions, yielding six experiment configurations per AGR. The best result in each configuration is shown in **bold**.

Mitigation	Late policy Malicious Activity Metric	10 %		20 %		30 %	
		LBIR	MMER	LBIR	MMER	LBIR	MMER
FedAvg	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
TrMean	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
Multi-KRUM	Benign	0.0 %	100.0 %	26.2 %	100.0 %	30.2 %	100.0 %
	Malicious		100.0 %		100.0 %		100.0 %
DNC	Benign	100.0 %	100.0 %	100.0 %	50.0 %	100 %	33.3 %
	Malicious		100.0 %		50.0 %		33.3 %
EE-TrMean	Benign	100.0 %	84.0 %	100.0 %	60.0 %	100.0 %	0.0 %
	Malicious		81.0 %		66.6 %		0.0 %

Table 4.2: Label-flipping – IID – Balanced: LBIR and MMER for each experiment in Table 4.1. Because every configuration contains malicious clients, MMER is reported for both policies. LBIR is meaningful only when benign clients join late, so the LBIR cells are left blank in the malicious-late rows. The best result in each configuration is shown in **bold**.

Table 4.1 shows the test accuracy and AUC of Label-Flipping attack when the class distribution is IID across clients and their volume is balanced. The label-flipping (LF) attack exerted only a modest influence on the global model accuracy. Resulting in all aggregation rules (AGRs) yielding test accuracies around 90% when 10% and 20% of the clients are malicious (Table 4.1). The overarching high test accuracies of the global models show that an honest majority in an homogeneous data scenario, is able to dominate the malicious updates, and pull the global model towards convergence. However at 30% malicious activity, we can see the effect of the Label flipping attack, where a majority of the AGRs report accuracies between 70% and 80%. Only one of the AGRs stands out—Multi-KRUM, which seemingly excels in this IID label distributed and volume balanced setting, even with a high degree of malicious activity. The stable model accuracy across the 6 experiments, can be explained by the 3 nearest neighbor rule of Multi-KRUM, which uses the 3 most similar updates in the aggregation of the new global model. The result shows that Multi-KRUM is able to filter the malicious model updates, when the data is homogeneous. Furthermore since the label distribution is IID and data volume is balanced, aggregating only 3 model updates is enough to achieve a high accuracy.

Table 4.2 shows the the LBIR and MMER of the AGRs for the experiments in Table 4.1. Although the overall performance band is narrow, the AGRs differ significantly in their treatment of late-joining clients. FedAvg admits every update and classical Trimmed-Mean partially admits every update it receives, benign or malicious. Consequently, both methods fail to fully exclude any attacker updates and once the proportion of malicious clients rises to 30%, their accuracies decline to the lower edge of the observed range ($\approx 74\%$ – 79%). This outcome exposes a limitation of the coordinate-level trimming of TrMean: partial weight vectors from malicious clients can propagate to the global model, at least for smaller trim factors, such as the 10% used in this thesis.

Divide-and-Conquer (DNC) achieved a favorable security vs. utility tradeoff, given that f was set to 1 throughout the thesis. Meaning that DNC filters only one client update each round. It both achieved some of the highest accuracies across this test set. Furthermore it also achieved high LBIR and a MMER of 100% for 10% malicious activity, 50% MMER for 20% malicious activity and 33.3% for 30% malicious activity. These figures precisely correspond to DNC only excluding malicious updates, since at the current hyperparameter setting, it is only able to filter out 1 client update per round. For example, for the 20% malicious activity cases, the maximum

MMER DNC can achieve, with $f=1$, is 50% (1 of 2 malicious updates) and analogously for the 30% malicious activity cases, a maximum MMER is at 33.3% (1 of 3 malicious updates). The use of SVD in DNC is seemingly a very effective measurement to identify and exclude label flipping attackers.

Finally, the adaptive EE-TrMean shows quite intriguing results for 10% and 20% malicious activity, achieving both high accuracies and high MMER, while achieving a high LBIR. However when the malicious client proportion rises to 30%, the ϵ -greedy is too permissive. The confounding factors are that the alpha-ramp rises too slowly and that the alpha maximum value is set too low, in order for it to be able to remove malicious clients from the exploit client set. Furthermore with the training horizon of only 25 rounds, the algorithm does not accumulate sufficient evidence to remove malicious clients, and MMER falls to zero.

4.2.2 Label flipping - IID Imbalanced

Mitigation	Late Policy Malicious Activity	Accuracy			AUC		
		10 %	20 %	30 %	10 %	20 %	30 %
FedAvg	Benign	43.2 %	40.5 %	44.5 %	67.405	68.727	75.241
	Malicious	43.2 %	42.6 %	42.5 %	67.036	69.175	69.879
TrMean	Benign	47.7 %	46.3 %	44.2 %	53.891	53.498	54.755
	Malicious	47.5 %	46.1 %	50.0 %	53.111	52.829	47.992
Multi-KRUM	Benign	52.9 %	53.6 %	53.7 %	49.508	48.942	50.298
	Malicious	53.4 %	53.6 %	53.6 %	49.180	48.942	48.942
DNC	Benign	52.1 %	52.8 %	48.2 %	50.197	49.439	59.060
	Malicious	53.0 %	53.1 %	52.3 %	49.154	48.456	49.585
EE-TrMean	Benign	52.0 %	51.5 %	48.4 %	50.922	49.855	51.865
	Malicious	52.2 %	52.6 %	48.6 %	49.690	50.019	52.681

Table 4.3: Label-flipping – IID – Imbalanced: Test Accuracy and Test Loss AUC, computed over all participating clients. Each AGR results is shown in two rows, corresponding to the two late-policies. Columns separate the results at the three malicious proportions, yielding six experiment configurations per AGR. The best result in each configuration is shown in **bold**.

Mitigation	Late policy Malicious Activity Metric	10 %		20 %		30 %	
		LBIR	MMER	LBIR	MMER	LBIR	MMER
FedAvg	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
TrMean	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
Multi-KRUM	Benign	33.3 %	100.0 %	0.0 %	100.0 %	33.3 %	100.0 %
	Malicious		100.0 %		100.0 %		100.0 %
DNC	Benign	100.0 %	100.0 %	100.0 %	50.0 %	100.0 %	29.3 %
	Malicious		100.0 %		50.0 %		33.3 %
EE-TrMean	Benign	100.0 %	72.0 %	100.0 %	56.0 %	100.0 %	22.7 %
	Malicious		85.7 %		57.1 %		15.9 %

Table 4.4: Label-flipping – IID – Imbalanced: LBIR and MMER for each experiment in Table 4.3. Because every configuration contains malicious clients, MMER is reported for both schedules. LBIR is meaningful only when benign clients join late, so the LBIR cells are left blank in the malicious-late rows. The best result in each configuration is shown in **bold**.

Table 4.3 shows the effect of label-flipping when the class distribution is IID across clients but their sample counts follow a Dirichlet draw. Relative to the IID, size-balanced baseline (Table 4.1) every AGR loses more than forty percentage points, and since the results show a quite steady test accuracy between the 3 attack densities, the results indicate that volume imbalance, decreases the global model performances significantly.

In this volume imbalanced setting, the naive FedAvg is outperformed by the robust AGRs. TrMean also performs quite poorly, just a few percentage points better than the naïve FedAvg baseline, and in most experiment cells TrMean under-performs the other robust AGRs. Multi-KRUM recorded the highest accuracy in all six experiment cells ($\approx 52\% - 54\%$). DNC also achieved relatively high accuracies, trailing Multi-KRUM by roughly one percentage point.

Table 4.4 shows the the LBIR and MMER of the AGRs when the class distribution is IID across clients and their volume is imbalanced. Multi-KRUM shows a minimal spread between late benign and late malicious schedules, as seen in Table 4.4. These figures corroborate the algorithm’s strength when benign updates form a dense cluster in gradient space. However the LBIR for Multi-KRUM never exceeds 33%. DNC included all late benign clients (LBIR = 100%), and similarly to the experiments in the IID balanced setting, reached the upper limit of MMER possible in the 10% and 20% malicious-client proportions, and coming very close to that limit in the 30% malicious activity experiments.

EE-TrMean improves on the classical TrMean results and achieved accuracies close to that of the best performing Multi-KRUM in the IID imbalanced setting. Compared with classical Trimmed-Mean, EE-TrMean gains $\approx 4\% - 5\%$ across the grid, while achieving relatively high LBIR and MMER across the experiment set. EE-TrMean scored similarly to DNC throughout the experiments, both in terms of LBIR, MMER and final test accuracy.

4.2.3 Label flipping - non-IID Partially Imbalanced

Mitigation	Late Policy Malicious Activity	Accuracy			AUC		
		10 %	20 %	30 %	10 %	20 %	30 %
FedAvg	Benign	62.1 %	63.3 %	60.0 %	37.414	39.476	41.517
	Malicious	68.3 %	59.6 %	58.9 %	30.392	42.624	39.874
TrMean	Benign	61.6 %	64.3 %	59.5 %	38.759	39.409	43.476
	Malicious	65.1 %	64.8 %	59.0 %	36.852	37.283	39.784
Multi-KRUM	Benign	58.7 %	58.6 %	42.1 %	43.037	43.773	51.465
	Malicious	58.7 %	58.7 %	59.1 %	42.714	42.477	40.841
DNC	Benign	62.5 %	63.6 %	62.6 %	38.196	37.869	40.937
	Malicious	62.1 %	65.1 %	62.0 %	37.515	36.441	38.521
EE-TrMean	Benign	59.4 %	58.6 %	53.9 %	40.193	41.027	45.367
	Malicious	62.7 %	61.0 %	55.3 %	40.088	35.537	40.678

Table 4.5: Label-flipping – non-IID – Partially Imbalanced: Test Accuracy and Test Loss AUC, computed over all participating clients. Each AGR results is shown in two rows, corresponding to the two late-policies. Columns separate the results at the three malicious proportions, yielding six experiment configurations per AGR. The best result in each configuration is shown in **bold**.

Mitigation	Late policy Malicious Activity Metric	10 %		20 %		30 %	
		LBIR	MMER	LBIR	MMER	LBIR	MMER
FedAvg	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
TrMean	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
Multi-KRUM	Benign	76.2 %	100.0 %	50.0 %	100.0 %	33.3 %	26.7 %
	Malicious		100.0 %		100.0 %		100.0 %
DNC	Benign	100.0 %	96.0 %	100.0 %	50.0 %	100.0 %	32.0 %
	Malicious		95.2 %		50.0 %		33.3 %
EE-TrMean	Benign	100.0 %	72.0 %	100.0 %	44.0 %	100.0 %	18.7 %
	Malicious		33.3 %		45.2 %		19.0 %

Table 4.6: Label-flipping – non-IID – Partially Imbalanced: LBIR and MMER for each experiment in Table 4.5. Because every configuration contains malicious clients, MMER is reported for both schedules. LBIR is meaningful only when benign clients join late, so the LBIR cells are left blank in the malicious-late rows. The best result in each configuration is shown in **bold**.

Table 4.5 shows the results when the class distribution is non-IID across clients and their sample count is partially imbalanced. At 10% malicious activity, FedAvg achieved the highest accuracy of the AGRs, especially looking at the late malicious setting, where it achieved roughly 3% more accuracy than the next best performing one—TrMean. Interestingly, DNC, which filters 1 update every round, does not achieve a higher accuracy than the naive FedAvg in this setting. At 20% malicious activity, all AGRs seem to achieve a similar accuracy to the baseline FedAvg, with DNC and TrMean achieving the best results. However the variability in accuracies between the AGRs is reduced ($\approx 5\%$ - 6%). Table 4.6, shows the LBIR and MMER of the AGRs for the experiments in Table 4.5. EE-TrMean and DNC, achieved a similar MMER of about 50%, while 100% LBIR. DNC achieved a slightly higher accuracy than EE-TrMean, possibly due to the fact that EE-TrMean continues to trim parameters, even when it has excluded clients. The continued parameter trimming could result in it removing honest-but-different updates, that contain valuable information for the global model to learn.

At 30% malicious activity, the best performing AGR is DNC, roughly matching the accuracies of the 10% and 20% malicious activity experiments. Multi-KRUM, by contrast, plunges to 42.1% accuracy in the late benign setting of 30% malicious activity experiments. To understand why, it is crucial to understand how the experiment is set up. In the late benign experiment on 30% malicious activity, only 7 clients, comprised of 4 benign clients and 3 malicious clients, are connected between rounds 1-4. Resulting that during the first rounds of the experiment, the malicious activity is not limited to 30%, but is instead approximately 42% (3 of 7 clients). Additionally all malicious clients run the same attack, so their updates can be expected to be similar. Thus their reduced pair-wise distance makes Multi-KRUM more likely to select them. However the results do indicate that benign clients are selected and aggregated later in the session, since the models do learn, but at a slower pace than in the late malicious client scenario, where rounds 1-4 are free of malicious activity. EE-TrMean shows quite poor performance, dropping ($\approx 5\%$ - 6%) accuracy compared to the classical TrMean. It achieved a MMER of $\approx 20\%$ in both late scenarios, but it does so while also excluding benign client indexed 1. This client's data partition is missing a few labels (Table 1 in the appendix). The missing labels, could be the reason why EE-TrMean falsely identifies it as a malicious client, since the resulting model update can be expected to differ from that of the majority of the model updates. However it might also be the case that benign client 1 contains valuable information that is lost when it is filtered out, and thus decreases the accuracy.

4.2.4 Label flipping - non-IID Imbalanced

Mitigation	Late Policy Malicious Activity	Accuracy			AUC		
		10 %	20 %	30 %	10 %	20 %	30 %
FedAvg	Benign	42.8 %	39.5 %	39.9 %	46.367	52.158	52.702
	Malicious	42.7 %	39.6 %	39.5 %	46.065	51.786	51.892
TrMean	Benign	58.7 %	53.0 %	53.9 %	35.690	40.796	39.842
	Malicious	58.3 %	58.6 %	59.3 %	35.228	37.228	37.788
Multi-KRUM	Benign	35.2 %	35.2 %	31.5 %	53.471	53.471	60.635
	Malicious	35.2 %	35.2 %	31.5 %	40.611	53.471	60.635
DNC	Benign	70.5 %	67.2 %	65.1 %	30.966	35.614	41.835
	Malicious	68.9 %	65.5 %	65.8 %	32.336	34.960	33.052
EE-TrMean	Benign	60.7 %	55.3 %	46.4 %	35.475	39.785	46.150
	Malicious	60.9 %	53.5 %	54.7 %	34.955	39.264	38.397

Table 4.7: Label-flipping – non-IID – Imbalanced: Test Accuracy and Test Loss AUC, computed over all participating clients. Each AGR results is shown in two rows, corresponding to the two late-policies. Columns separate the results at the three malicious proportions, yielding six experiment configurations per AGR. The best result in each configuration is shown in **bold**.

Mitigation	Late policy Malicious Activity Metric	10 %		20 %		30 %	
		LBIR	MMER	LBIR	MMER	LBIR	MMER
FedAvg	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
TrMean	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
Multi-KRUM	Benign	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %	66.6 %
	Malicious		100.0 %		100.0 %		66.6 %
DNC	Benign	100.0 %	28.0 %	100.0 %	44.0 %	92.1 %	21.3 %
	Malicious		52.4 %		45.2 %		33.3 %
EE-TrMean	Benign	100.0 %	48.0 %	100.0 %	42.0 %	80.1 %	26.7 %
	Malicious		76.2 %		45.2 %		22.2 %

Table 4.8: Label-flipping – non-IID – Imbalanced: LBIR and MMER for each experiment in Table 4.7. Because every configuration contains malicious clients, MMER is reported for both schedules. LBIR is meaningful only when benign clients join late, so the LBIR cells are left blank in the malicious-late rows. The best result in each configuration is shown in **bold**.

Table 4.7 shows how volume imbalanced and non-IID label distributions is a challenging configuration in FL. The FedAvg baseline, which in previous experiment sets often matched or only performed slightly worse than the robust AGRs, now achieved a test accuracy of roughly 40%. The decline shows that statistical heterogeneity is sufficient to slow down convergence or decrease model performance, poisoning merely exacerbates the problem. The relatively small effect of label flipping can be seen by the relatively small decrease in accuracy between the 10% malicious activity experiments and the 30% malicious activity experiments of the FedAvg baseline.

In this setting, a weakness in Multi-KRUM becomes clear, its reliance on selecting a subset of trusted clients in the aggregation of the global models, narrows the update diversity and hinders the model's ability to learn. Because Multi-KRUM aggregates only the 3 most central updates each round, it effectively trains on under and over-represented labeled data partitions. In a highly imbalanced, non-IID setting these partitions can be dominated by majority classes, which the model quickly learns to over-fit to, while minority classes which are under-represented become under-fitted. Accuracy therefore falls even though Multi-KRUM excludes most malicious clients. By contrast Divide-and-Conquer achieved the best accuracies for this experiment set across the board, indicating that DNC is robust when the data heterogeneity is high.

Table 4.8 shows the corresponding LBIR and MMER of the experiments in Table 4.7. DNC is able to achieve a high LBIR in a data-heterogeneous scenario, where honest client updates are expected to differ substantially between each other. Looking at the MMER, DNC also correctly excludes malicious clients, varying between 52.4 % and 21.3 % in the three different scenarios. Illustrating that the use of SVD to find malicious updates, is effective against Label Flipping even when the data is highly heterogeneous.

EE-TrMean achieved better accuracy than classical TrMean, and does so while fully excluding malicious and including late benign updates, showing that light exploration vs. exploitation can boost classic coordinate-trimming under severe data heterogeneity.

4.2.5 Little-is-Enough - IID Imbalanced

Mitigation	Late Policy Malicious Activity	Accuracy			Loss AUC		
		10 %	20 %	30 %	10 %	20 %	30 %
FedAvg	Benign	44.9 %	10.1 %	31.2 %	51.837	78.032	59.213
	Malicious	45.8 %	29.5 %	43.1 %	53.392	64.668	54.874
TrMean	Benign	53.6 %	32.0 %	36.7 %	46.956	63.753	56.220
	Malicious	53.7 %	41.3 %	45.1 %	46.979	53.814	51.045
Multi-KRUM	Benign	58.2 %	54.2 %	39.2 %	45.229	46.189	53.154
	Malicious	63.1 %	57.3 %	58.4 %	29.838	44.430	45.206
DNC	Benign	53.2 %	42.1 %	45.1 %	49.001	54.996	52.867
	Malicious	53.3 %	43.8 %	42.6 %	49.115	53.891	53.740
EE-TrMean	Benign	53.9 %	49.5 %	48.4 %	47.175	53.277	49.352
	Malicious	54.7 %	51.3 %	52.4 %	46.769	48.423	47.352

Table 4.9: Little-is-Enough – IID – Imbalanced: Test Accuracy and Test Loss AUC, computed over all participating clients. Each AGR results is shown in two rows, corresponding to the two late-policies. Columns separate the results at the three malicious proportions, yielding six experiment configurations per AGR. The best result in each configuration is shown in **bold**.

Mitigation	Late policy Malicious Activity Metric	10 %		20 %		30 %	
		LBIR	MMER	LBIR	MMER	LBIR	MMER
FedAvg	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
TrMean	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
Multi-KRUM	Benign	100.0 %	48.0 %	50.0 %	76.0 %	54.0 %	82.7 %
	Malicious		38.1 %		76.2 %		81.0 %
DNC	Benign	100.0 %	32.0 %	100.0 %	16.0 %	100.0 %	13.3 %
	Malicious		33.3 %		16.6 %		12.7 %
EE-TrMean	Benign	100.0 %	76.0 %	100.0 %	62.0 %	100.0 %	29.3 %
	Malicious		90.5 %		45.2 %		25.4 %

Table 4.10: Little-is-Enough – IID – Imbalanced: LBIR and MMER for each experiment in Table 4.9. Because every configuration contains malicious clients, MMER is reported for both schedules. LBIR is meaningful only when benign clients join late, so the LBIR cells are left blank in the malicious-late rows. The best result in each configuration is shown in **bold**.

Table 4.9 shows the effect of Little-is-Enough, when the class distribution is IID across clients but their sample counts follow a Dirichlet draw. For 10% malicious activity, FedAvg has roughly 10% less accuracy than the second worst performing AGR– DNC. However most of the AGRs have a similar test accuracy, with only Multi-KRUM achieving a higher accuracy than the other robust AGRs.

In the 20% malicious activity case, the decline of the FedAvg baseline becomes even more severe, while DNC and TrMean also drop quite significantly. Only EE-TrMean and Multi-KRUM achieved relatively high accuracies. Moreover in the 30% malicious activity cases, Multi-KRUM once again drops quite significantly in the late benign case, with an accuracy of 39.2%, now with an accuracy comparable to the other robust AGRs. This mirrors the results seen earlier, using Label Flipping on non-IID label distribution and partially imbalanced volume (Table 4.5). The accuracy of EE-TrMean on the other hand, does not drop significantly, but only slightly decreases as the malicious activity increases.

Table 4.10 shows the LBIR and MMER of the AGRs when the class distribution is IID across clients, but their volume is imbalanced. At 10% malicious activity all of the AGRs allow all late honest clients, however with varying degrees of success in excluding malicious clients. EE-TrMean achieved the best MMER at 10% malicious activity of 90.5% and 76% MMER, in the late benign and late malicious cases respectively. These high rates indicate that EE-TrMean is able to quickly and accurately identify the malicious client and remove it from the aggregation. In the 20% malicious activity cases, Multi-KRUM achieved the best MMER, but it also excluded one of the late honest clients (seen from the fact that it achieved 50% LBIR, with 2 late benign clients). EE-TrMean once again achieved a quite high MMER in both cases and 100% LBIR in the late benign case.

In the 30% malicious activity cases, Multi-KRUM again achieved the highest MMER, while EE-TrMean, achieved a 100% LBIR in the late benign case and 25.4% and 29.3% MMER in the late malicious and late benign scenario respectively. DNC managed to accurately exclude malicious clients one third of the rounds in the 10% malicious activity case, corresponding to the attack frequency of LIE on every third round, resulting in quite robust filtering against LIE. Correspondingly in the 20% malicious activity case, it achieved a MMER of almost one sixth, once again reaching close to the maximum exclusion rate possible using $f=1$. However still resulting in quite poor global model test accuracy.

4.2.6 Little-is-Enough - non-IID Partially Imbalanced

Mitigation	Late Policy Malicious Activity	Accuracy			AUC		
		10 %	20 %	30 %	10 %	20 %	30 %
FedAvg	Benign	49.2 %	43.4 %	50.4 %	46.237	48.960	46.206
	Malicious	51.3 %	48.8 %	53.0 %	44.036	45.183	42.893
TrMean	Benign	58.9 %	50.9 %	49.8 %	38.460	44.315	46.071
	Malicious	59.1 %	52.4 %	52.4 %	38.034	41.978	42.966
Multi-KRUM	Benign	57.5 %	58.5 %	57.4 %	41.797	40.990	43.894
	Malicious	57.5 %	57.5 %	56.7 %	41.797	41.896	43.755
DNC	Benign	58.1 %	54.7 %	52.4 %	38.197	42.218	44.875
	Malicious	59.7 %	53.9 %	54.0 %	38.047	41.469	41.515
EE-TrMean	Benign	61.7 %	58.1 %	52.9 %	38.314	43.131	44.901
	Malicious	60.8 %	60.5 %	56.2 %	38.105	41.114	42.148

Table 4.11: Little-is-Enough – non-IID – Partially Imbalanced: Test Accuracy and Test Loss AUC, computed over all participating clients. Each AGR results is shown in two rows, corresponding to the two late-policies. Columns separate the results at the three malicious proportions, yielding six experiment configurations per AGR. The best result in each configuration is shown in **bold**.

Mitigation	Late policy Malicious Activity Metric	10 %		20 %		30 %	
		LBIR	MMER	LBIR	MMER	LBIR	MMER
FedAvg	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
TrMean	Benign	100.0 %	0.0 %	100.0 %	0.0 %	100.0 %	0.0 %
	Malicious		0.0 %		0.0 %		0.0 %
Multi-KRUM	Benign	95.2 %	60.0 %	52.4 %	72.0 %	31.7 %	77.3 %
	Malicious		52.4 %		81.0 %		77.8 %
DNC	Benign	100.0 %	32.0 %	92.9 %	20.0 %	87.3 %	16.0 %
	Malicious		33.3 %		16.7 %		12.7 %
EE-TrMean	Benign	95.2 %	60.0 %	100.0 %	26.0 %	100.0 %	16.0 %
	Malicious		52.4 %		30.1 %		14.3 %

Table 4.12: Little-is-Enough – non-IID – Partially Imbalanced: LBIR and MMER. Because every configuration contains malicious clients, MMER is reported for both schedules. LBIR is meaningful only when benign clients join late, so the LBIR cells are left blank in the malicious-late rows. The best result in each configuration is shown in **bold**.

Table 4.11 shows the results of the different AGRs during Little-is-Enough attack, when the class distribution is non-IID across clients and their sample is partially imbalanced. At 10% malicious activity all the robust AGRs achieve roughly 10% better accuracy than the baseline FedAvg. EE-TrMean achieved the highest accuracy, with two or three percentage points to the second best performing. However looking at the AUC, the differences are smaller and the small advantage in accuracy might not be due to any larger generalization of the global models.

In the 20% malicious activity case, the best performing AGRs are now Multi-KRUM and EE-TrMean, both achieving accuracies of roughly 60% in both the benign and malicious late policy. A lead of roughly 5% better test accuracy than the third best performing one– DNC. At 30% malicious activity Multi-KRUM achieved the best accuracies and also a stable accuracy in relation to the previous two degrees of malicious activity. The stable accuracy of Multi-KRUM, across the 6 experiments, is due to the fact that it only selects three clients for aggregation and in the 6 different experiments performed in the non-IID and partially imbalanced setting, it seemingly selects a similar constitution of clients each time, thus achieving a similar accuracy.

Table 4.12 shows the LBIR and MMER of the AGRs when the class distribution is non-IID across clients and their volume is partially imbalanced. At 10% malicious activity, all of the AGRs achieve a high LBIR in the late benign case. In terms of MMER, Multi-KRUM and EE-TrMean achieved the same MMER while DNC, accurately excludes one third of the rounds, corresponding to the attack frequency of Little-is-Enough every third round. At 20% malicious activity only the naive FedAvg, TrMean and EE-TrMean achieved full inclusion of the late benign clients, while Multi-KRUM achieved a significantly higher MMER than the remaining AGRs. Finally in the 30% malicious activity cases, Multi-KRUM achieved a similar MMER as in the 20% malicious activity case, but with an even lower LBIR than before.

4.2.7 Experiments and Research Questions

Across the six experiment sets we examined whether (i) established robust aggregation rules (Multi-KRUM, Trimmed-Mean, Divide-and-Conquer) and (ii) our adaptive rule (EE-TrMean) can separate honest from malicious clients when the federations face non-IID data, volume imbalance, and late participation.

Using four partitioning schemes, two late-arrival policies, three attack intensities, and two attack types, we isolated one factor at a time. This design lets us track each AGRs strengths and weaknesses as the data heterogeneity changes (RQ2) and to what extent EE-TrMean maintains accuracy and resilience under the same conditions (RQ3).

The numerical outcomes are presented in Sections 4.1–4.6 and their implications for RQ2 and RQ3 are discussed in the forthcoming Discussion chapter.

Chapter 5

Discussion

This thesis set out to design a scalable, Kubernetes-orchestrated simulator for adversarial federated learning (FL), to benchmark the resilience of state-of-the-art aggregation rules under realistic conditions, and to propose an adaptive alternative that relaxes fixed and hard threshold heuristics. The experimental suite comprised of 180 distinct experiments in which two attacks: Label-Flipping and Little-Is-Enough, were used on four data-partition regimes, two participation schedules, three attacker densities and 5 aggregation rules.

5.1 Research Question 1: Realism and Scalability of the Simulator

The simulator satisfied the two goals of realism and reproducibility. Horizontal scaling of clients is easily achieved through a single line change, while stateless pods ensures fault tolerance. Kubernetes was chosen due to its widespread use as a container orchestration tool in industry as well as in academia— thus point of using Kubernetes was to increase the usability of the simulator. Figures 28 and 29 in the appendix shows a linear CPU and memory usage when the client number increases, showing that the simulator is scalable and deterministic in its use of CPU and memory per client. Thus scaling the client count can easily be achieved by calculating the CPU and memory usage per client and then multiplying it by the number of clients requested, then scaling the cluster resources based on the peak resources consumed by the simulator. Furthermore by supporting late joining, heterogeneous data partitioning, insertion of new attacks and defense modules, the multi-node simulator fulfills the realism, modularity and pluggability goals of this project.

In light of these features and the substantial experiment suite performed in this thesis, Research Question 1 can be answered affirmatively: the test-bed is both realistic and reusable by other researchers.

5.2 Research Question 2: Behaviour of Existing Robust Aggregators

The experimental results paint a nuanced picture of current SOTA mitigation strategies. Multi-KRUM yielded high and stable accuracy in some experiments, but its performance could deteriorate when the data heterogeneity was pronounced. With $k = 3$, the server aggregates only three client updates per round, in a non-IID, imbalanced federation the global model is unlikely to be trained equally for every label each round, so it tends to over-fit dominant classes and leave minority ones under-fitted, resulting in poor test accuracy. This sensitivity to k illustrates why static thresholds can undermine Multi-KRUM's robustness under realistic and skewed partitions. Coordinate-wise Trimmed-Mean never managed to exclude a malicious client completely, nevertheless, it mitigated the degradation of performance observed in FedAvg in many of the experiments. Divide-and-Conquer showed a good safety vs. utility tradeoff, given the use of $f=1$, which filters only one client update each round (Figure 15 in the appendix). Though DNC filters only a single adversary at a time, it did so quite reliably. Important to note is that we did not investigate the optimal parameters for either of the robust aggregation strategies. Multi-KRUM could possibly have performed better with a larger k . A similar argument could be made for Divide-and-Conquer, setting f to 2 (or 3) would most likely have improved the results, especially in terms of MMER for DNC. However these results reflect how the algorithms would be used in practice, when the user would not prior to an attack know what level of malicious activity to expect. Thus the static parameters of Multi-KRUM, TrMean and DNC need careful tuning in order to avoid; either using too few contributions, such as the case with Multi-KRUM, or being too permissive, such as the case with DNC. Adaptive strategies that tune thresholds could therefore cut down on the needed manual calibration while preserving robustness. Finally our findings reinforce previous warnings that IID-only benchmarks overstate the real-world strength of classic defenses [4].

5.3 Research Question 3: Effectiveness of EE–Trimmed-Mean

The proposed Exploration–Exploitation Trimmed-Mean combines coordinate trimming with an ε -greedy trust score and an alpha ramp. At an attacker share of ten percent, EE-TrMean showed signs of correctly excluding malicious clients. Although it could at times fail to achieve a high MMER, due to the hyper-parameters used in the thesis. EE-TrMean did however achieve both a high LBIR and a relatively high accuracy throughout the experiments. Indicating that a soft threshold adaptive strategy is a promising path forward, that does not require parameter tuning of the static rules that Multi-KRUM, DNC and TrMean use.

EE-TrMean occasionally removed honest-but-different clients (see Appendix Figure 5). In one experiment, benign client 1 was excluded. One explanation to why it was excluded is that benign client 1 did not contain all labels and therefore its update differed from the majority. Furthermore the late malicious client schedule highlights a weakness in EE-TrMean. During the rounds 1–4 no attacker participated, yet EE-TrMean still updated trust scores. Resulting in some benign clients accumulating low scores early. When adversaries finally joined, those already down-voted clients could not score enough to re-enter the aggregation. We observed this pattern in some scenarios where the malicious clients appeared late. Figures 16 and 17 (appendix) show that the chosen α -ramp and ε -greedy settings were sub-optimal in the 80-round experiments. Because the ramp slope is scaled by the total session length, and extending the session from 25 to 80 rounds stretched the ramp, the malicious client was excluded only at round 60. As a result, malicious clients lingered far longer than in the shorter experiments. Scaling α by a fixed length instead of the full session length, would likely have removed the malicious clients sooner and thus resulted in better performance.

In summation, the results still indicate that a soft exploration can improve on the static rules of most SOTA AGRs, paving way for further work in this direction.

Chapter 6

Conclusions and Future Work

The research confirms that no single aggregation rule suffices across the all of statistical and adversarial scenarios. The naive averaging of FedAvg, works well in many of the scenarios, and the level of data heterogeneity causes much more damage to the model performance than the two attacks. Multi-KRUM remains a strong baseline under mild collusion, but can falter under specific conditions, and finding an optimal value of k , is crucial to it's ability to perform when the data heterogeneity is high. Coordinate-wise defenses such as Trimmed-Mean provides only a partial protection, but still improves the accuracy to that of the FedAvg baseline. DNC, proves to have a robust rule for detecting malicious updates and achieved some of the best accuracies in the experiments. However DNC still requires finding and setting an optimal value of f to exclude malicious updates and include honest ones. The proposed EE-Trimmed-Mean builds on TrMean by enabling to fully reject malicious updates, while still allowing for late benign clients to participate in the aggregation. However, EE-TrMean did at times achieve a lower accuracy than the classical TrMean because it excluded honest-but-different updates. Beyond the algorithmic contribution, the open-source simulator is available to the community with a replicable platform on which future defenses and attacks can be tested in a reproducible and realistic manner.

6.1 Limitations

The study is bounded by a few limitations. The training proceeded for only twenty-five rounds. Only 2 experiment sets with 80 rounds were executed. In the data heterogeneous scenarios, the models had yet to converge (Figures: 8, 10, 12, 14, 16 in the appendix). Furthermore the homogeneous latency of

Google Cloud does not reflect the stochastic jitter of mobile networks or poor connectivity. Resulting in no dropouts during training, which is a frequent reality in FL. Furthermore, the attacks comprised of only two static poisoning strategies, Label Flipping and Little-is-Enough. Finally, given that this work was done during one semester, the time constraint was severe, both learning the tools needed to complete the goals, building the framework, generating results and finally finishing the thesis report.

6.2 Unaddressed Questions

Mitigation and attacks for larger models have not been explored. Another point, which has not been explored is how the parameter settings of the mitigation strategies affect the results, to test them both at one uniform setting across the test series, and also perform a hyperparameter scan to find optimal values for different scenarios. Furthermore another topic left unexplored is intermittent client connectivity, which is quite common in FL. Where clients might participate late, but also exit early. Lastly, other types of attacks, such as backdoor attacks or inference attacks have not been evaluated in this thesis, which are also important questions to address.

6.3 Next Steps and Future Work

Given that DNC performed quite well in terms of; global model accuracy, excluding malicious updates and including honest but late updates, a concrete next step would be add a similar exploration vs exploitation strategy to DNC. Given that DNC had a high degree of success both in terms of MMER and LBIR, EE-DNC is an interesting and obvious next step of this thesis. Another interesting avenue of work is optimizing AGRs to not only exclude malicious client updates, but also improve learning when the data is heterogeneous. Data volume heterogeneity and non-IID label distributions caused significantly more harm to the global models accuracies than any of the attacks. Consequently the results also speak for the general robustness of FL networks against adversarial attacks. At times FedAvg only dropped a few percentage points accuracy between the three attack densities, while dropping between 30%-50% accuracy between the different data partitioning schemes. Interestingly, under some conditions the robust aggregation methods lost more test-accuracy than naïve FedAvg, for instance Multi-KRUM suffered a significant drop in accuracy in the non-IID, imbalanced partitioning setting.

Another avenue of future work is testing further RL techniques to optimize the AGRs in allowing honest-but-different clients and excluding malicious clients, such as utilizing Thompson sampling. Bandit methods, such as Thompson sampling, could adjust each client aggregation weight, using validation accuracy as the reward to balance exploration and exploitation

6.4 Reflections

Conducting this extensive work under time constraints required significant planning and work ethic. From the start, this was clear, which drove a rapid development of the multi-node simulator. Container orchestration and Kubernetes imposed steep learning curves, which yet resulted in disciplined work. Ethically, strengthening FL defenses not only shields collaborative models from sabotage but also bolsters societal trust in decentralized learning, thereby lessening the incentive to centralize sensitive data, and improving data privacy. The thesis should therefore be viewed as a foundation rather than a final destination, a reproducible springboard from which the community can iterate, stress-test, and hopefully improve upon.

References

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 54, Fort Lauderdale, FL, USA, 2017, [Online]. Available: <https://arxiv.org/abs/1602.05629>. [Pages 1, 9, and 11.]
- [2] M. A. Khan, V. Shejwalkar, A. Houmansadr, and F. M. Anwar, “On the pitfalls of security evaluation of robust federated learning,” in *IEEE Symposium on Security & Privacy Workshops*, 2023. [Pages 3 and 8.]
- [3] M. Ekmefjord, A. Ait-Mlouk, S. Alawadi, M. Åkesson, P. Singh, O. Spjuth, S. Toor, and A. Hellander, “Scalable federated machine learning with fedn,” *arXiv preprint arXiv:2103.00148*, Apr 2022, [Online]. Available: <https://arxiv.org/abs/2103.00148>. [Pages 10, 13, 14, and 26.]
- [4] V. Shejwalkar, A. Houmansadr, P. Kairouz, and D. Ramage, “Back to the drawing board: A critical evaluation of poisoning attacks on production federated learning,” in *2022 IEEE Symposium on Security and Privacy (SP)*, San Francisco, CA, USA, 2022. doi: 10.1109/SP46214.2022.00065 pp. 1349–1366. [Pages 10, 14, 16, 17, 33, 38, and 68.]
- [5] S. H. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. U. Stich, and A. T. Suresh, “Adaptive gradient methods for federated learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5915–5927, [Online]. Available: <https://arxiv.org/abs/2003.00295>. [Pages 11 and 12.]
- [6] L. Lyu, H. Yu, X. Ma, C. Chen, L. Sun, J. Zhao, Q. Yang, and P. S. Yu, “Privacy and robustness in federated learning: Attacks and defenses,”

- IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 7, pp. 8726–8737, Jul 2024. doi: 10.1109/TNNLS.2022.3216981 [Pages 14 and 17.]
- [7] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, “Analyzing federated learning through an adversarial lens,” *arXiv preprint arXiv:1811.12470*, Nov 2019, [Online]. Available: <https://arxiv.org/abs/1811.12470>. [Pages 14, 15, 26, 33, and 39.]
- [8] R. Ramakrishna and G. Dán, “Inferring class-label distribution in federated learning,” in *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security (AISec)*. Association for Computing Machinery, 2022. doi: 10.1145/3560830.3563725 pp. 1–12. [Page 14.]
- [9] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, “Inverting gradients: How easy is it to break privacy in federated learning?” in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 16 937–16 947. [Online]. Available: https://papers.nips.cc/paper_files/paper/2020/file/c4ede56bbd98819ae6112b20ac6bf145-Paper.pdf [Page 14.]
- [10] E. Baruch, M. Baruch, and Y. Goldberg, “A little is enough: Circumventing defenses for distributed learning,” *arXiv preprint arXiv:1902.06156*, 2019, [Online]. Available: <https://arxiv.org/abs/1902.06156>. [Pages 15, 16, 26, and 33.]
- [11] Z. Sun, P. Kairouz, A. T. Suresh, and H. B. McMahan, “Can you really backdoor federated learning?” *arXiv preprint arXiv:1911.07963*, 2019, [Online]. Available: <https://arxiv.org/abs/1911.07963>. [Pages 17 and 18.]
- [12] Dua, Dheeru and Graff, Casey, “UCI machine learning repository: Iris data set,” <https://archive.ics.uci.edu/dataset/53/iris>, 2019, accessed: Apr. 3, 2025. [Page 18.]
- [13] “Flower: A friendly federated learning framework,” [Online]. Available: <https://flower.ai>, Flower, 2025, accessed: Apr. 3, 2025. [Page 18.]
- [14] “Hugging face hub (online repository of models and datasets),” [Online]. Available: <https://huggingface.co>, Hugging Face, 2025, accessed: Apr. 3, 2025. [Page 18.]

- [15] D. Seneviratne, “Impact evaluation of poisoning attacks in federated learning scenarios,” Master’s Thesis, Uppsala University, Uppsala, Sweden, 2024. [Pages 18, 26, 27, 35, 38, and 39.]
- [16] D. Yin, Y. Chen, K. Ramchandran, and P. L. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. Stockholm, Sweden: PMLR, 10–15 Jul. 2018, pp. 5666–5674. [Online]. Available: <https://proceedings.mlr.press/v80/yin18a.html> [Page 20.]
- [17] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine-tolerant gradient descent,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, Long Beach, CA, USA, 2017, pp. 119–129, [Online]. Available: <https://arxiv.org/abs/1703.02757>. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/file/f4b9ec30ad9f68f89b29639786cb62ef-Paper.pdf> [Pages 20, 22, and 23.]
- [18] E. M. El Mhamdi, R. Guerraoui, and S. Rouault, “The hidden vulnerability of distributed learning in byzantium,” *arXiv preprint arXiv:1802.07927*, 2018. [Online]. Available: <https://arxiv.org/abs/1802.07927> [Page 20.]
- [19] V. Shejwalkar and A. Houmansadr, “Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning,” in *Proceedings of the Network and Distributed Systems Security (NDSS) Symposium 2021*, Virtual, 2021. doi: 10.14722/ndss.2021.24498 [Pages 20, 23, 24, 26, and 43.]
- [20] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018, [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>. [Page 24.]
- [21] S. Bubeck and N. Cesa-Bianchi, “Regret analysis of stochastic and nonstochastic multi-armed bandit problems,” *Foundations and Trends in Machine Learning*, vol. 5, no. 1, pp. 1–122, 2012. doi: 10.1561/22000000024 [Pages 24 and 25.]

- [22] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time analysis of the multiarmed bandit problem,” *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002. doi: 10.1023/A:1013689704352 [Page 24.]
- [23] D. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, “A tutorial on thompson sampling,” *Foundations and Trends in Machine Learning*, vol. 11, no. 1, pp. 1–96, 2018. doi: 10.1561/22000000070 [Page 25.]
- [24] W. R. Thompson, “On the likelihood that one unknown probability exceeds another in view of the evidence of two samples,” *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933. doi: 10.2307/2332286 [Page 25.]
- [25] S. Agrawal and N. Goyal, “Analysis of thompson sampling for the multi-armed bandit problem,” in *Proceedings of the 25th Annual Conference on Learning Theory (COLT)*, 2012, pp. 39.1–39.26, [Online]. Available: <http://proceedings.mlr.press/v23/agrawal12/agrawal12.pdf>. [Page 25.]
- [26] S. Caldas *et al.*, “Leaf: A benchmark for federated settings,” *arXiv preprint arXiv:1812.01097*, 2019, [Online]. Available: <https://arxiv.org/abs/1812.01097>. [Page 26.]
- [27] H. N. Cunha Neto, J. Hribar, I. Dusparic, N. C. Fernandes, and D. M. F. Mattos, “Fedsbs: Federated-learning participant-selection method for intrusion detection systems,” *Computer Networks*, vol. 244, p. 110351, 2024. doi: 10.1016/j.comnet.2024.110351. [Online]. Available: <https://doi.org/10.1016/j.comnet.2024.110351> [Page 27.]
- [28] H. Huang, J. Yang, Y. Chen, J. Ye, and D. Wu, “Fedrts: Federated robust pruning via combinatorial thompson sampling,” *arXiv*, 2025, version 2, 17 May 2025. [Page 28.]



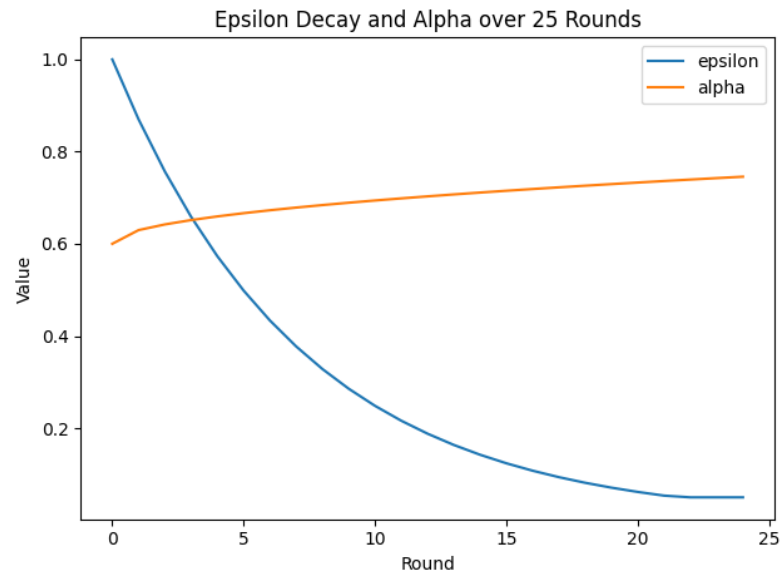


Figure 1: Epsilon Decay and Alpha ramp-up

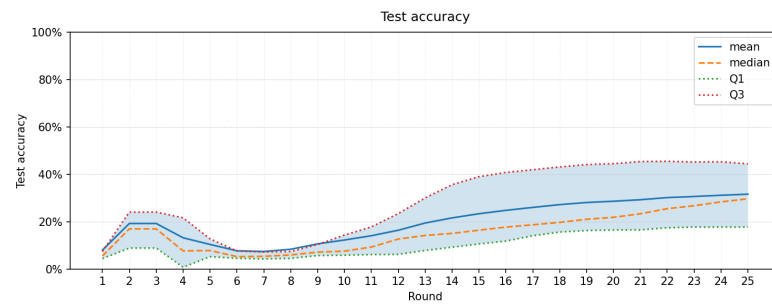


Figure 2: Multi-KRUM - non-IID - Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping Test Accuracy

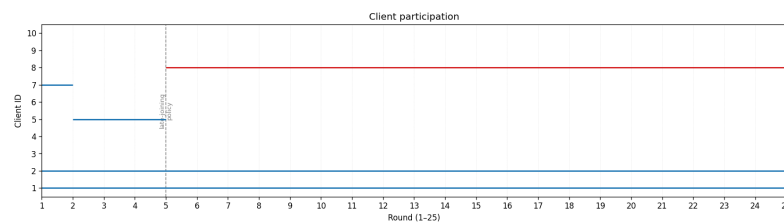


Figure 3: Multi-KRUM - non-IID - Imbalanced - 3 Malicious Clients - 3 Malicious Clients 5 rounds late - Label Flipping - Timeline

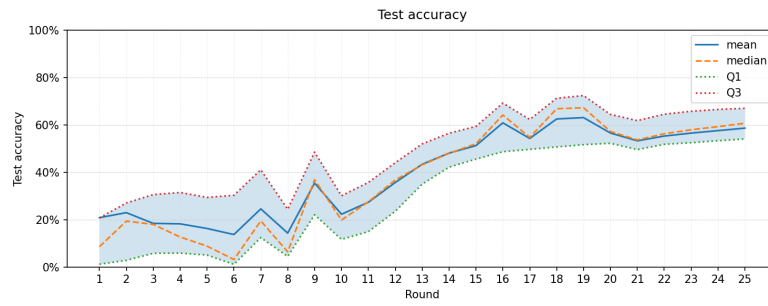


Figure 4: EETrMean - non-IID - Partially Imbalanced - 2 Malicious clients - 2 Benign clients 5 rounds late - Label Flipping - Test Accuracy

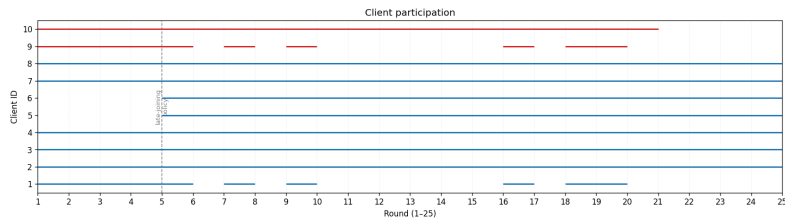


Figure 5: EETrMean - non-IID - Partially Imbalanced - 2 Malicious clients - 2 Benign Clients 5 rounds late - Label Flipping - Timeline

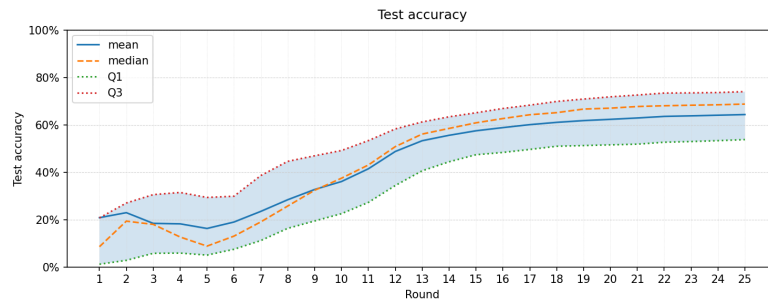


Figure 6: TrMean - non-IID - Partially Imbalanced - 2 Malicious clients - 2 Benign clients 5 rounds late - Label Flipping - Test Accuracy

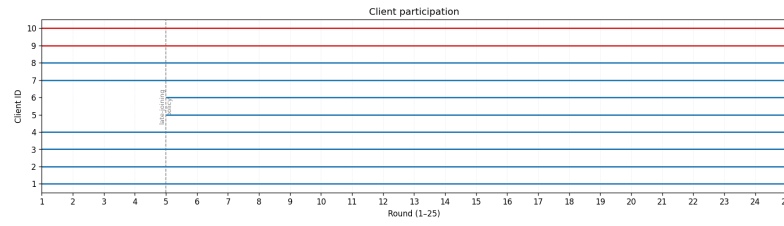


Figure 7: TrMean - non-IID - Partially Imbalanced - 2 Malicious clients - 2 Benign Clients 5 rounds late - Label Flipping - Timeline

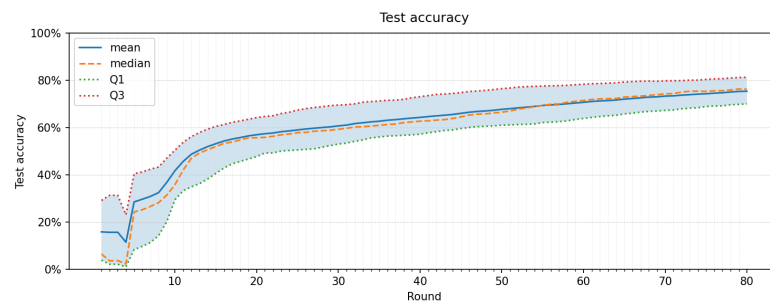


Figure 8: Fedavg - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds

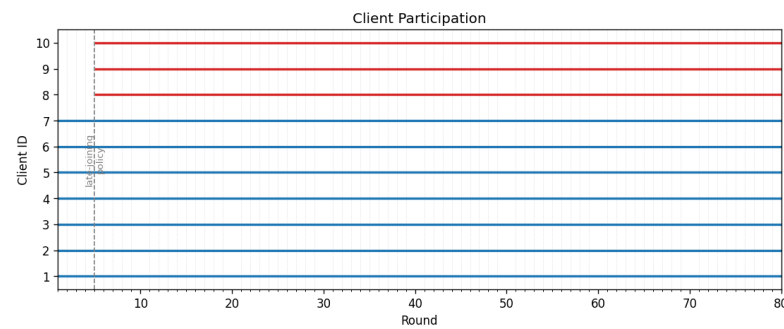


Figure 9: Fedavg - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds

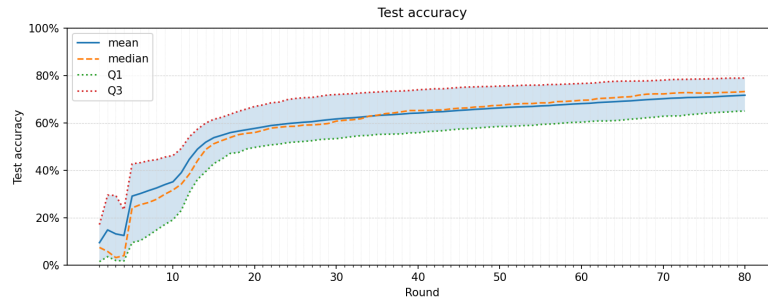


Figure 10: TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds

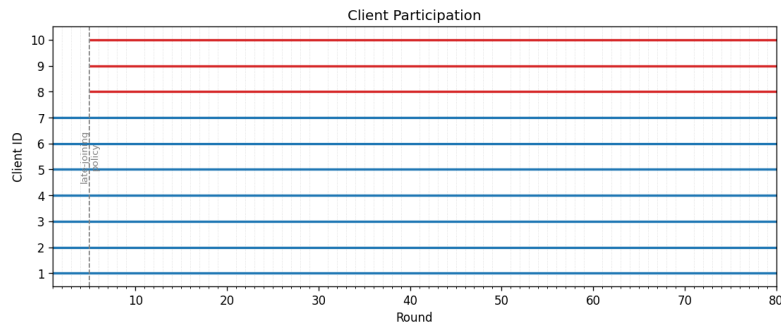


Figure 11: TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds

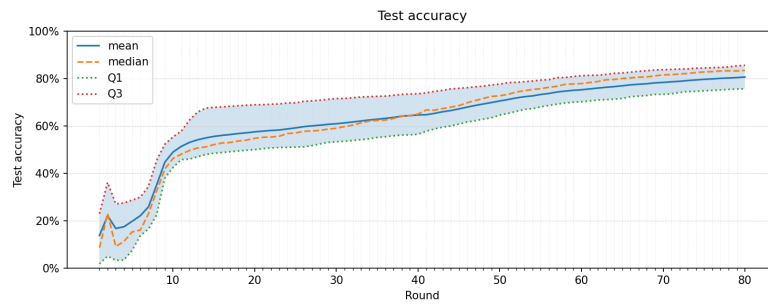


Figure 12: Multi-KRUM - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds

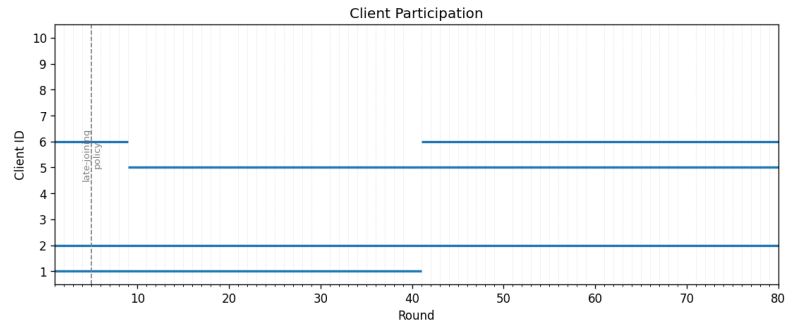


Figure 13: Multi-KRUM - non-IID - Partially Imbalanced - 3 Malicious clients
- 3 Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds

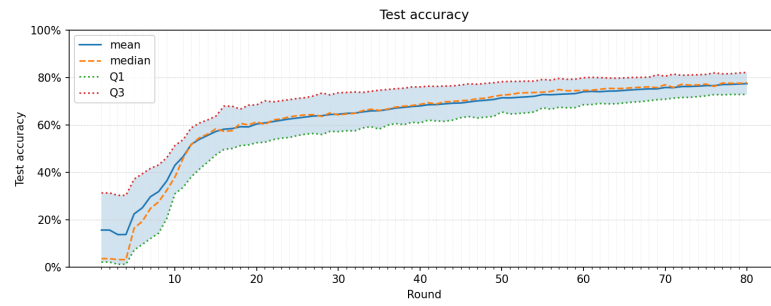


Figure 14: DNC - non-IID - Partially Imbalanced - 3 Malicious clients - 3
Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds

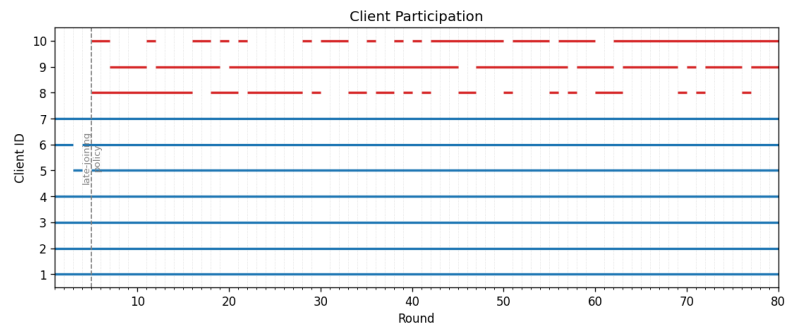


Figure 15: DNC - non-IID - Partially Imbalanced - 3 Malicious clients - 3
Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds

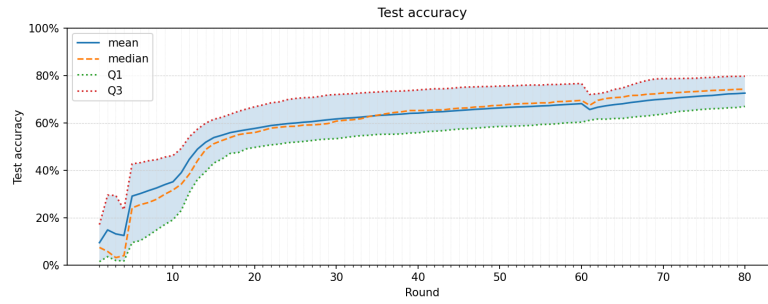


Figure 16: EE-TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Test Accuracy - 80 rounds

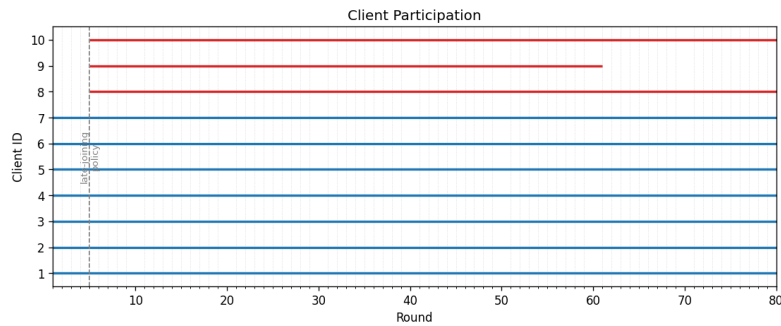


Figure 17: EE-TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Label Flipping - Timeline - 80 rounds

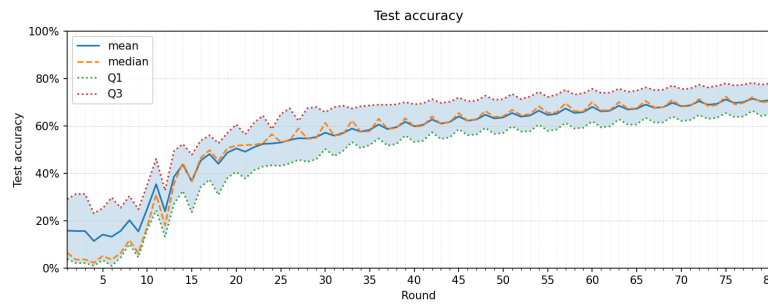


Figure 18: Fedavg - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds

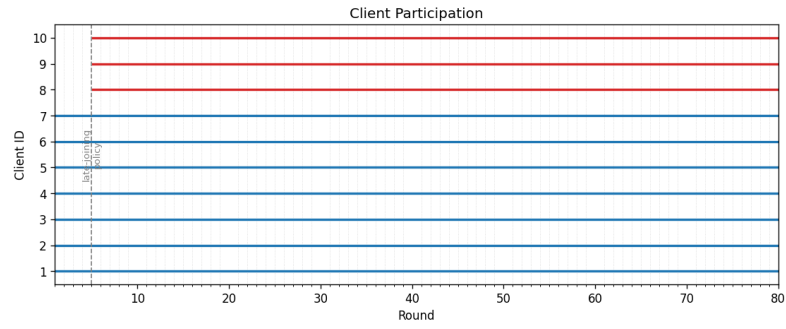


Figure 19: Fedavg - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds

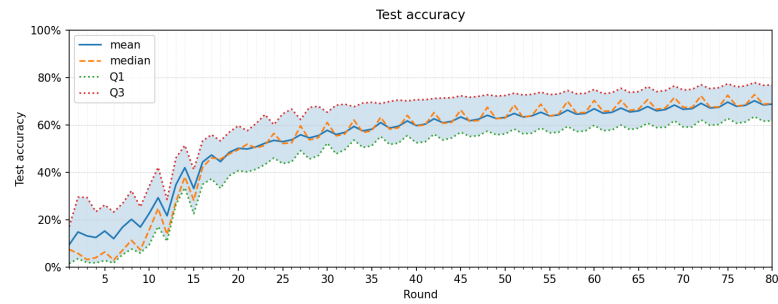


Figure 20: TrMean - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds

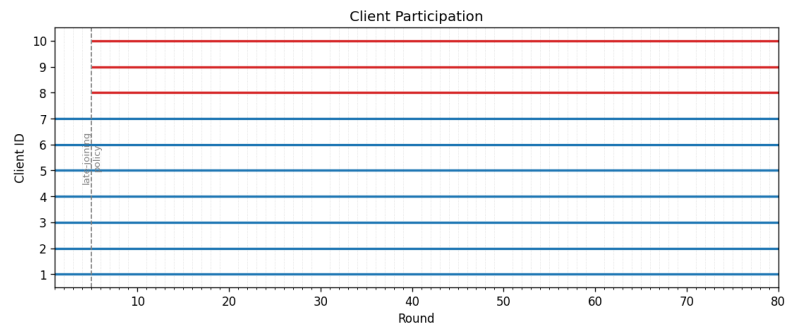


Figure 21: TrMean - non-IID - Partially Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds

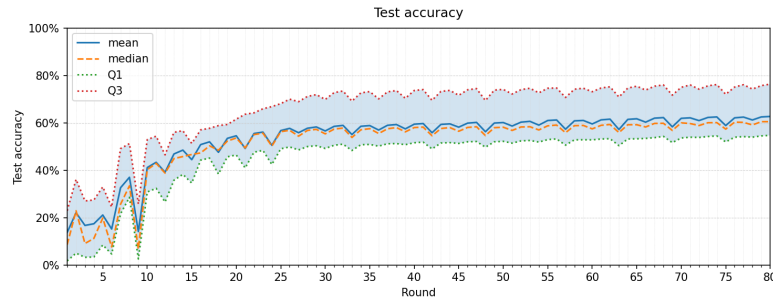


Figure 22: Multi-KRUM - non-IID - Partially Imbalanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds

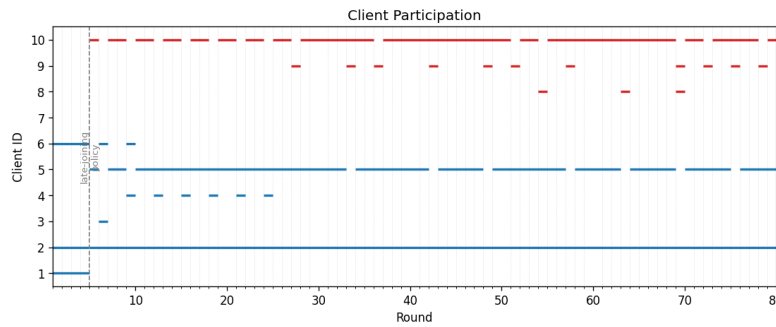


Figure 23: Multi-KRUM - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds

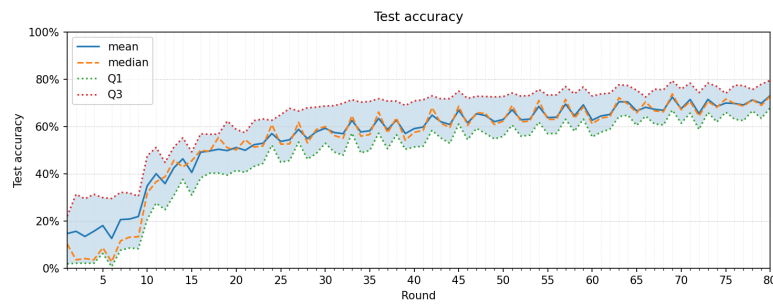


Figure 24: DNC - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds

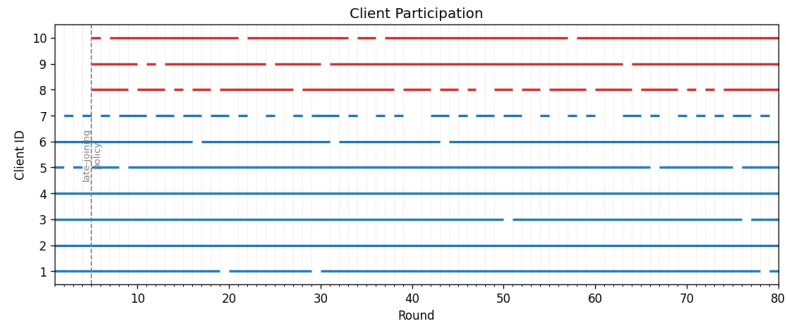


Figure 25: DNC - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds

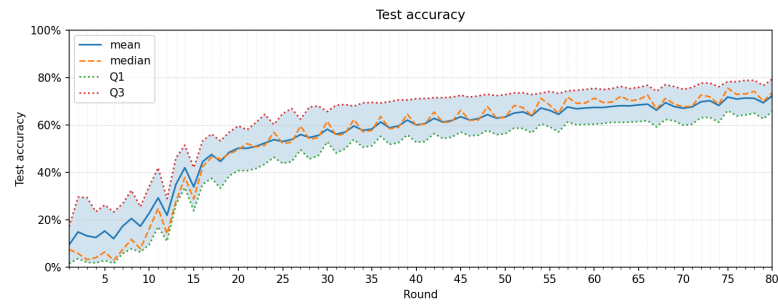


Figure 26: EE-TrMean - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Test Accuracy - 80 rounds

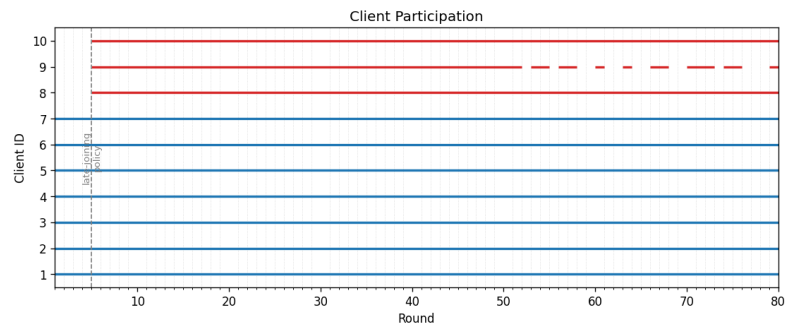


Figure 27: EE-TrMean - non-IID - Balanced - 3 Malicious clients - 3 Malicious clients 5 rounds late - Little-is-Enough - Timeline - 80 rounds

Partition	Client	0	1	2	3	4	5	6	7	8	9	N
IID, balanced	1	0.099	0.112	0.097	0.101	0.104	0.086	0.101	0.108	0.092	0.100	6000
	2	0.102	0.113	0.099	0.103	0.094	0.089	0.100	0.105	0.096	0.099	6000
	3	0.096	0.117	0.094	0.109	0.094	0.090	0.094	0.104	0.101	0.102	6000
	4	0.100	0.114	0.101	0.101	0.100	0.093	0.100	0.098	0.093	0.101	6000
	5	0.097	0.115	0.100	0.098	0.096	0.094	0.101	0.103	0.097	0.099	6000
	6	0.096	0.115	0.099	0.103	0.097	0.087	0.099	0.102	0.100	0.104	6000
	7	0.102	0.114	0.101	0.096	0.099	0.092	0.102	0.102	0.095	0.098	6000
	8	0.098	0.112	0.102	0.104	0.093	0.090	0.095	0.106	0.102	0.099	6000
	9	0.101	0.107	0.099	0.102	0.099	0.091	0.100	0.105	0.100	0.095	6000
	10	0.098	0.105	0.100	0.104	0.099	0.092	0.095	0.111	0.100	0.096	6000
IID, Imbalanced	1	0.099	0.112	0.099	0.102	0.097	0.091	0.099	0.104	0.097	0.099	1612
	2	0.099	0.112	0.099	0.102	0.098	0.090	0.099	0.105	0.098	0.099	830
	3	0.099	0.112	0.099	0.102	0.097	0.090	0.099	0.104	0.097	0.099	12524
	4	0.099	0.112	0.099	0.102	0.097	0.090	0.099	0.104	0.097	0.099	7787
	5	0.099	0.112	0.099	0.102	0.097	0.090	0.099	0.104	0.098	0.099	4734
	6	0.099	0.112	0.099	0.102	0.097	0.090	0.099	0.104	0.098	0.099	9675
	7	0.099	0.112	0.099	0.102	0.097	0.090	0.099	0.104	0.098	0.099	7680
	8	0.098	0.112	0.099	0.103	0.097	0.090	0.098	0.105	0.097	0.099	975
	9	0.099	0.112	0.099	0.102	0.097	0.090	0.099	0.104	0.098	0.099	10523
	10	0.099	0.112	0.099	0.102	0.097	0.090	0.099	0.105	0.098	0.099	3654
Non-IID, Partially Imbalanced	1	0.196	0.150	0.003	0.263	0.000	0.107	0.070	0.163	0.044	0.004	6000
	2	0.285	0.012	0.000	0.000	0.066	0.193	0.039	0.198	0.098	0.109	6000
	3	0.029	0.000	0.074	0.087	0.007	0.058	0.642	0.051	0.003	0.049	6000
	4	0.009	0.048	0.037	0.287	0.007	0.010	0.166	0.110	0.025	0.302	6000
	5	0.017	0.403	0.033	0.005	0.013	0.076	0.077	0.232	0.134	0.011	5445
	6	0.014	0.050	0.025	0.001	0.168	0.555	0.000	0.054	0.001	0.132	3774
	7	0.229	0.068	0.391	0.047	0.091	0.065	0.000	0.000	0.000	0.108	5554
	8	0.211	0.043	0.005	0.080	0.133	0.104	0.000	0.133	0.290	0.001	3331
	9	0.116	0.002	0.142	0.002	0.201	0.000	0.000	0.013	0.464	0.060	5950
	10	0.000	0.527	0.044	0.041	0.028	0.000	0.000	0.262	0.086	0.012	4352
Non-IID, Imbalanced	1	0.023	0.068	0.210	0.042	0.434	0.060	0.110	0.004	0.040	0.011	1613
	2	0.011	0.000	0.080	0.267	0.281	0.025	0.000	0.065	0.269	0.002	829
	3	0.380	0.046	0.089	0.063	0.078	0.000	0.000	0.105	0.236	0.001	12524
	4	0.030	0.021	0.022	0.153	0.009	0.184	0.060	0.033	0.340	0.148	7664
	5	0.004	0.075	0.000	0.217	0.010	0.656	0.012	0.018	0.000	0.008	4720
	6	0.013	0.030	0.235	0.058	0.057	0.059	0.001	0.259	0.000	0.289	9657
	7	0.032	0.266	0.040	0.093	0.463	0.032	0.064	0.002	0.000	0.009	7045
	8	0.031	0.155	0.321	0.227	0.000	0.000	0.030	0.114	0.000	0.122	907
	9	0.090	0.565	0.005	0.010	0.000	0.000	0.101	0.228	0.000	0.000	5463
	10	0.000	0.009	0.251	0.064	0.000	0.000	0.672	0.003	0.000	0.001	1786

Table 1: Per-client class proportions (columns #0–#9) and total sample counts (N) under the four dataset-partition strategies used in our federated experiments: IID–balanced, IID–Imbalanced, non-IID - Partially Imbalanced, and non-IID–Imbalanced. Each block lists the ten clients (rows), making clear how class balance and client data volume vary across the different partitioning schemes.

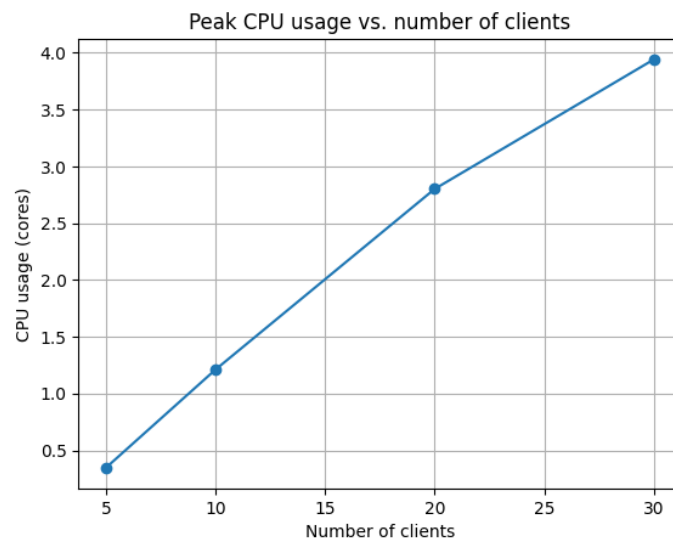


Figure 28: Peak CPU consumption as the client count increases.

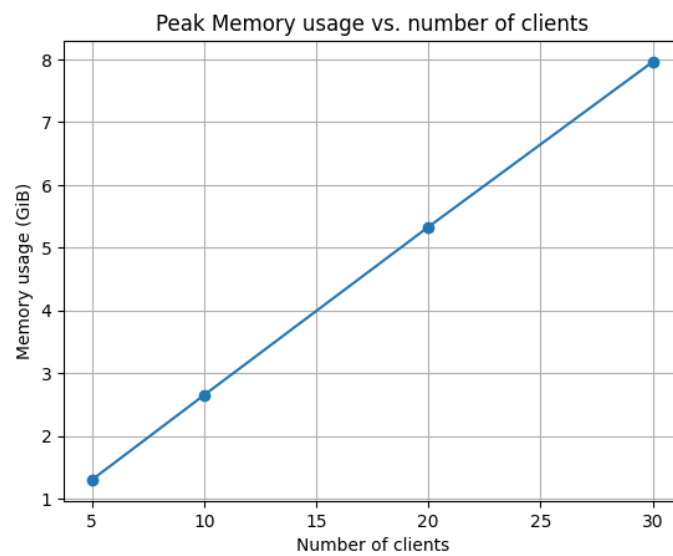


Figure 29: Peak memory consumption as the client count increases.

€€€€ For DIVA €€€€

```
{
  "Author1": { "Last name": "Dackevall",
    "First name": "Sigvard",
    "Local User Id": "u100001",
    "E-mail": "sigvardd@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
    }
  },
  "Cycle": "2",
  "Course code": "DA231X",
  "Credits": "30.0",
  "Degree1": { "Educational program": "Degree Programme in Computer Science and Engineering",
    "programcode": "CDATE",
    "Degree": "Degree of Master of Science in Engineering",
    "subjectArea": "Computer Science and Engineering"
  },
  "Title": {
    "Main title": "Multi-Node Attack and Defense Toolkit for Federated Learning",
    "Subtitle": "Exploration of Adaptive Aggregation Strategies",
    "Language": "eng" },
    "Alternative title": {
      "Main title": "Konstruktion av ett Multi-Nod attack och försvars-Toolkit för Federerad Inläring",
      "Subtitle": "Utforskning av adaptiva aggregeringsmetoder",
      "Language": "swe"
    },
  },
  "Supervisor1": { "Last name": "Byrd Victorica",
    "First name": "Mauricio",
    "Local User Id": "u100003",
    "E-mail": "mbv@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
      "L2": "Computer Science" }
  },
  "Supervisor3": { "Last name": "Toor",
    "First name": "Salman",
    "E-mail": "salman@scaleoutsystems.com",
  },
  "Examiner1": { "Last name": "Dán",
    "First name": "György",
    "Local User Id": "u1d13i2c",
    "E-mail": "gyuri@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
      "L2": "Computer Science" }
  },
  "Cooperation": { "Partner_name": "Scaleout Systems AB",
    "National Subject Categories": "10299, 102",
    "Other information": { "Year": "2025", "Number of pages": "xvi,89",
    "Copyrightleft": "copyright",
    "Series": { "Title of series": "TRITA – EECS-EX", "No. in series": "2024:0000" },
    "Opponents": { "Name": "A. B. Normal & A. X. E. Normalè",
    "Presentation": { "Date": "2022-03-15 13:00"
    },
    "Language": "eng"
    },
    "Room": "via Zoom https://kth-se.zoom.us/j/ddddddddddd",
    "Address": "Isafjordsgatan 22 (Kistagången 16)",
    "City": "Stockholm",
    "Number of lang instances": "2",
    "Abstract[eng ]": €€€€
    €€€€,
    "Keywords[eng ]": €€€€
    Dockerbehållare, Kubernetes, Aggregation regel (AGR), Federerad inläring (FL), Combiner, Federated Average (FedAvg), Trimmed Mean (TrMean), KRUM, Multi-KRUM, Divide-and-Conquer (DnC) €€€€,
    "Abstract[swe ]": €€€€
    €€€€,
    "Keywords[swe ]": €€€€
    €€€€,
  }
}
```

acronyms.tex

```
%%% Local Variables:
%%% mode: latex
%%% TeX-master: t
%%% End:
% The following command is used with glossaries-extra
\setabbreviationstyle[acronym]{long-short}
% The form of the entries in this file is \newacronym[label]{acronym}{phrase}
%                                     or \newacronym[options]{label}{acronym}{phrase}
% see "User Manual for glossaries.sty" for the details about the options, one example is shown below
% note the specification of the long form plural in the line below
%
% The following example also uses options
\newacronym[shortplural={OSes}, firstplural={operating systems (OSes)}]{OS}{OS}{Operating System}

\newacronym{KTH}{KTH}{KTH Royal Institute of Technology}

\newacronym{agr}{AGR}{Aggregation Rule}

\newacronym{dlf}{DLF}{Dynamic Label Flipping (Attack)}

\newacronym{dnc}{DnC}{Divide-and-Conquer}

\newacronym{fedadam}{FedAdam}{Federated Adam}

\newacronym{fedavg}{FedAvg}{Federated Averaging}

\newacronym{fedn}{FEDn}{FL framework from Scaleout Systems }

\newacronym{femnist}{FEMNIST}{Federated Extended MNIST}

\newacronym{fl}{FL}{Federated Learning}

\newacronym{git}{Git}{Version control software}

\newacronym{github}{GitHub}{Online repository}

\newacronym{grpc}{gRPC}{Google Remote Procedure Call}

\newacronym{helm}{Helm}{A package manager for Kubernetes}

\newacronym{iid}{IID}{Independent and Identically Distributed}

\newacronym{iot}{IoT}{Internet of Things}

\newacronym{iris}{IRIS}{A classic small dataset}

\newacronym{krum}{KRUM}{A robust aggregator}

\newacronym{kubernetes}{Kubernetes (K8s)}{A container orchestration platform}

\newacronym{lie}{LIE}{A Little Is Enough (Attack)}

\newacronym{mnist}{MNIST}{Modified National Institute of Standards and Technology (Dataset)}

\newacronym{multi-krum}{Multi-Krum}{A robust aggregator}

\newacronym{non-iid}{non-IID}{Non-Independent and Identically Distributed}

\newacronym{pytorch}{PyTorch}{A popular deep learning framework}

\newacronym{s3}{S3}{A cloud storage mechanism}

\newacronym{slf}{SLF}{Static Label Flipping (Attack)}

\newacronym{svd}{SVD}{Singular Value Decomposition}

\newacronym{tensorflow}{TensorFlow}{A popular deep learning framework}

\newacronym{trmean}{TrMean}{Trimmed Mean (A robust aggregator)}

\newacronym{vm}{VM}{Virtual Machine}
```